



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»
Образовательный центр 806
Группа М8О-401Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»
Профиль Информатика
Квалификация: бакалавр

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

На тему: Система управления молочным животноводством на базе доильных роботов, совместимых с Lely Astronaut

Автор ВКРБ: Кабанов Антон Алексеевич ()
Руководитель: Ухов Петр Александрович ()
Консультант: — ()
Консультант: — ()
Рецензент: — ()

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ()
 мая 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 83 страниц, 25 рисунков, 11 таблиц, 48 использованных источников, 4 приложений.

ИНФОРМАЦИОННАЯ СИСТЕМА, МОЛОЧНАЯ ФЕРМА, RUST, SVELTEKIT, МАШИННОЕ ОБУЧЕНИЕ, ПРЕДИКТИВНАЯ АНАЛИТИКА, ПРОГНОЗИРОВАНИЕ НАДООВ, ОБНАРУЖЕНИЕ МАСТИТА, ИНТЕГРАЦИЯ LELY, DOCKER

Объектом исследования является процесс автоматизации управления молочной фермой. Предметом исследования — методы и средства разработки информационных систем управления молочными фермами с интеграцией роботизированного доильного оборудования и предиктивной аналитикой на основе моделей машинного обучения. Цель работы — разработка информационной системы, обеспечивающей комплексный учёт животных, надоев, воспроизводства, кормления и интеграцию с роботизированным доильным оборудованием Lely, а также модуля предиктивной аналитики на основе моделей машинного обучения.

Основное содержание работы состоит в проектировании и реализации трёхзвенной архитектуры: серверной части на Rust/Axum с JWT-аутентификацией, middleware безопасности и интеграцией с Lely Horizon; клиентского веб-приложения на SvelteKit с адаптивным интерфейсом; и модуля аналитики с 11 моделями машинного обучения на Python/FastAPI. Серверная часть содержит собственные алгоритмы прогнозирования (метод Хольта–Винтерса, кривая лактации Вуда, индекс здоровья) и систему автоматического сравнения 8 моделей с выбором оптимальной.

Основным результатом работы является информационная система, покрывающая полный цикл управления стадом: от учёта и мониторинга до предиктивной аналитики с доверительными интервалами и раннего обнаружения заболеваний. Система развёрнута в контейнерах Docker с мониторингом на базе стека Grafana LGTM и конвейером CI/CD на GitHub Actions.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	10
1 Анализ предметной области и техническое задание на разработку системы управления молочным животноводством	13
1.1 Точное животноводство и автоматизация управления молочными фермами	13
1.2 Обзор существующих решений	14
1.3 Техническое задание	16
1.4 Стекло технологий	18
2 Проектирование и разработка системы управления молочным животноводством	20
2.1 Архитектура системы	20
2.2 Модель данных	22
2.3 Серверная часть (Backend)	25
2.3.1 Маршрутизация и обработка запросов	26
2.3.2 Аутентификация и авторизация	27
2.3.3 Промежуточное программное обеспечение (Middleware)	27
2.3.4 Интеграция с Lely	28
2.4 Клиентское веб-приложение (Frontend)	29
2.4.1 Маршрутизация и навигация	29
2.4.2 Аутентификация на стороне клиента	30
2.4.3 API-клиент	30
2.4.4 Компоненты пользовательского интерфейса	30
2.4.5 Генерация отчётов	31
2.5 Модуль машинного обучения (Analytics-ML)	33
2.5.1 Выбор алгоритмов	33
2.5.2 Обоснование выбора метрик	35
2.5.3 Формирование обучающих меток	36
2.5.4 Прогноз надоев с доверительными интервалами	37
2.5.5 Инженерия признаков	37
2.5.6 MLOps: оптимизация, мониторинг дрейфа и управление моделями	38

2.6	Предиктивная аналитика в серверной части	39
2.6.1	Сравнение моделей прогнозирования временных рядов . .	40
2.6.2	Ансамблевый прогноз	42
2.6.3	Объяснимость прогнозов (SHAP)	43
2.7	Оптимизация производительности	44
2.7.1	Анализ узких мест	44
2.7.2	Оптимизация базы данных	44
2.7.3	Кеширование моделей машинного обучения	45
2.7.4	Интеграция колоночной базы данных ClickHouse	46
2.7.5	Оптимизация серверной части	47
2.7.6	Оптимизация клиентского приложения	48
2.7.7	Результаты оптимизации	49
2.8	Мониторинг	50
2.9	Непрерывная интеграция и доставка (CI/CD)	52
2.10	Развёртывание	53
3	Тестирование и демонстрация работы системы управления молочным животноводством	55
3.1	Стратегия тестирования	55
3.2	Тестирование серверной части	55
3.3	Тестирование клиентского приложения	56
3.4	Тестирование модуля машинного обучения	57
3.5	Нагрузочное тестирование	57
3.6	Демонстрация системы	58
3.6.1	Аутентификация	58
3.6.2	Дашборд	59
3.6.3	Управление животными	60
3.6.4	Отчёты и аналитика	62
3.6.5	Интеграция с Lely	63
3.6.6	Оповещения	63
3.7	Мониторинг работоспособности	63
	ЗАКЛЮЧЕНИЕ	65
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
	ПРИЛОЖЕНИЕ А Исходный код	71
	ПРИЛОЖЕНИЕ Б Описание таблиц базы данных	72
	ПРИЛОЖЕНИЕ В Визуализация модели данных	75

ПРИЛОЖЕНИЕ Г Примеры API-запросов и ответов	79
---	----

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Информационная система — совокупность аппаратных и программных компонентов, обеспечивающих сбор, хранение, обработку и выдачу информации для решения задач управления

REST API — архитектурный стиль взаимодействия распределённых компонентов сетевых приложений, основанный на протоколе HTTP и принципах REST

Машинное обучение — раздел искусственного интеллекта, изучающий методы построения моделей, способных обучаться на данных и делать прогнозы или принимать решения без явного программирования

Временной ряд — последовательность данных, собранных или измеренных в последовательные моменты времени через равные или неравные промежутки

Лактация — процесс образования и выделения молока молочной железой сельскохозяйственных животных

Охота (эструс) — период половой активности у самок млекопитающих, во время которого происходит овуляция и возможно осеменение

Мастит — воспалительное заболевание вымени коровы, вызываемое бактериальной инфекцией и приводящее к снижению качества и объёма молока

Кетоз — метаболическое нарушение у коров в ранний послеродовой период, характеризующееся повышенным содержанием кетоновых тел в крови

Выбраковка — процесс исключения животного из стада по производственным, ветеринарным или экономическим причинам

Z-оценка — стандартизованное отклонение значения от среднего: $z = (x - \mu) / \sigma$, где μ — среднее по стаду, σ — стандартное отклонение. Значения $|z| > 2$ считаются статистически аномальными

Индекс здоровья — комплексный показатель состояния животного, рассчитываемый как взвешенная сумма Z-оценок надоя, жвачки, активности и SCC. Принимает значения от 0 до 100, где ≥ 80 — низкий риск, < 40 — критический

Доверительный интервал — диапазон значений, в котором с заданной вероятностью находится истинное значение прогнозируемой величины. В

данной работе используются квантильные интервалы q_{10}/q_{90} , содержащие 80 процентов распределения прогнозов

SHAP — SHapley Additive exPlanations — метод объяснимости машинного обучения, основанный на значениях Шепли из теории кооперативных игр. Показывает вклад каждого признака в прогноз модели

Ансамбль моделей — метод объединения прогнозов нескольких моделей в один итоговый прогноз. В данной работе используется взвешенное среднее с весами, обратно пропорциональными MAPE каждой модели

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

ИС — информационная система

БД — база данных

СУБД — система управления базами данных

API — application programming interface

REST — representational state transfer

МО — машинное обучение

CI — continuous integration

CD — continuous delivery

JWT — JSON web token

HTTP — hypertext transfer protocol

HTTPS — HTTP secure

SSL — secure sockets layer

TLS — transport layer security

JSON — JavaScript object notation

CSV — comma-separated values

PDF — portable document format

UI — user interface

SCC — somatic cell count

BCS — body condition score

FPR — fat-protein ratio

DIM — days in milk

SSE — server-sent events

CORS — cross-origin resource sharing

SPA — single-page application

SSR — server-side rendering

GHCR — GitHub container registry

SRE — site reliability engineering

DOM — document object model

MAPE — mean absolute percentage error

RMSE — root mean squared error

SES — simple exponential smoothing

ARIMA — autoregressive integrated moving average

KMeans — k-means clustering

ВВЕДЕНИЕ

Актуальность темы исследования обусловлена тем, что молочное животноводство является одной из ключевых отраслей агропромышленного комплекса: по данным ФАО, мировое производство коровьего молока превышает 750 млн тонн в год и продолжает расти со среднегодовым темпом 1,5–2 процента [1]. Эффективность управления молочной фермой напрямую влияет на рентабельность производства: своевременное выявление мастита снижает экономические потери на 30–50 процентов, а оптимизация кормления позволяет повысить надои на 5–15 процентов [2].

Концепция точного животноводства (*precision livestock farming*), предложенная Д. Беркмансом в 2008 году, предполагает непрерывный автоматизированный мониторинг состояния животных с использованием датчиков и информационных технологий [1]. Современные молочные фермы оснащаются роботизированными доильными установками, датчиками активности и жвачки, автоматическими кормушками, генерирующими значительные объёмы данных — до 250 000 записей в сутки для стада в 500 голов [2]. Без специализированной информационной системы оперативная обработка этих данных невозможна, что приводит к запоздалому выявлению заболеваний, неоптимальному кормлению и потерям продукции.

Существующие системы управления молочными фермами (*Lely Horizon*, *DeLaval DelPro*, *SCR Heatime*, *1C: Управление фермой*) обладают существенными ограничениями: монолитная архитектура, затрудняющая расширение; отсутствие встроенных средств предиктивной аналитики; привязка к оборудованию одного производителя; закрытый API, не позволяющий внедрять собственные модели машинного обучения. Поэтому актуальна задача создания информационной системы, объединяющей полный цикл управления стадом, интеграцию с роботизированным оборудованием и модули машинного обучения в единой открытой платформе.

Объектом исследования является процесс автоматизации управления молочной фермой. Предметом исследования — методы и средства разработки информационных систем управления молочными фермами с интеграцией роботизированного доильного оборудования и предиктивной аналитикой на основе моделей машинного обучения. Цель работы — разработка информационной системы управления молочной фермой с интеграцией

роботизированного доильного оборудования Lely и предиктивной аналитикой на основе моделей машинного обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- а) провести анализ предметной области и существующих решений, на основании которого сформулировать техническое задание и обосновать выбор стека технологий;
- б) спроектировать трёхзвенную архитектуру и модель данных базы данных, содержащую более 35 таблиц с расширением TimescaleDB для работы с временными рядами;
- в) реализовать серверный API на Rust/Axum с аутентификацией, авторизацией, middleware безопасности и интеграцией с облачным API Lely Horizon;
- г) разработать клиентское веб-приложение и модуль машинного обучения с 11 моделями для предиктивной аналитики;
- д) реализовать серверные алгоритмы прогнозирования, ансамблевый метод и объяснимость прогнозов на основе SHAP;
- е) провести функциональное и нагрузочное тестирование системы, а также разработать конвейер CI/CD и мониторинг.

В работе использованы следующие методы исследования: системный анализ предметной области, проектирование реляционных баз данных с использованием расширенной модели “сущность–связь” и нормализации, методы машинного обучения (градиентный бустинг XGBoost с квантильной регрессией, кластеризация KMeans, обнаружение аномалий Isolation Forest), анализ временных рядов (экспоненциальное сглаживание Хольта–Винтерса, ряды Фурье, ARIMA, линейная регрессия), а также методы программной инженерии (контейнеризация Docker, непрерывная интеграция и доставка, мониторинг на базе стека Grafana LGTM). Качество моделей прогнозирования оценивается по метрикам MAPE и RMSE на тестовых выборках с хронологическим разбиением. Объяснимость прогнозов обеспечивается методом SHAP, позволяющим количественно оценить вклад каждого признака.

Научная новизна работы заключается в следующем:

- а) предложена архитектура информационной системы, интегрирующая серверные алгоритмы аналитики на Rust с внешним

модулем машинного обучения на Python, что обеспечивает работу предиктивной аналитики даже при недоступности ML-сервиса;

- б) разработан ансамблевый метод объединения прогнозов с весами, обратно пропорциональными MAPE, повышающий робастность прогнозирования;
- в) реализована система автоматического сравнения 8 моделей прогнозирования временных рядов с выбором оптимальной для каждого животного индивидуально.

Практическая значимость работы заключается в том, что разработанная информационная система может быть использована на молочных фермах, оснащённых роботизированным доильным оборудованием Lely, для автоматизации процессов учёта и мониторинга. Внедрение системы позволяет повысить эффективность управления стадом за счёт предиктивной аналитики, сократить время на ведение документации и своевременно выявлять заболевания животных на ранних стадиях.

Выпускная квалификационная работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе проведён анализ предметной области: рассмотрена концепция точного животноводства, выполнен обзор существующих решений и сформулировано техническое задание. Во второй главе описаны проектирование и разработка системы: архитектура, модель данных, серверная и клиентская части, модуль машинного обучения, серверная аналитика, мониторинг и развёртывание. В третьей главе представлены результаты тестирования и демонстрация работы системы. В заключении сформулированы основные результаты работы и перспективы дальнейшего развития.

1 Анализ предметной области и техническое задание на разработку системы управления молочным животноводством

1.1 Точное животноводство и автоматизация управления молочными фермами

Концепция точного животноводства (precision livestock farming, PLF) [1] предполагает непрерывный автоматизированный мониторинг состояния животных с использованием датчиков и информационных технологий для принятия управленческих решений в реальном времени. Данная концепция была предложена Д. Беркмансом и К. Беркманс в 2008 году и с тех пор получила широкое распространение в молочном животноводстве [2]. Согласно исследованию Европейской комиссии по сельскому хозяйству, внедрение PLF-технологий позволяет снизить использование антибиотиков на 20–30 процентов, повысить продуктивность стада на 10–15 процентов и сократить трудозатраты на мониторинг в 3–5 раз.

В молочном животноводстве PLF охватывает несколько ключевых направлений:

- а) мониторинг надоев — объём молока по четвертям вымени, скорость доения (кг/мин), электропроводность (мСм/см) как индикатор мастита, количество соматических клеток (SCC, тыс./мл). Типичная корова при доильном роботе Lely Astronaut генерирует 2–3 визита в сутки, каждый из которых содержит 15–20 параметров;
- б) мониторинг здоровья — активность (шаги за час), жвачка (минуты жевания за период), температура тела и индекс потребления корма. Датчики активности (Lely Qwes, SCR HR) фиксируют данные с дискретностью 1–2 часа, что позволяет обнаруживать отклонения в течение суток;
- в) управление воспроизводством — обнаружение охоты по активности (повышение в 3–5 раз относительно базовой линии), контроль стельности и планирование отёлов и запусков. Интервал между отёлами — ключевой KPI: целевое значение 365–400 дней, каждый лишний день обходится в 3–5 долларов потерь от непродуктивного содержания;
- г) управление кормлением — контроль рационов (сухое вещество,

netto-energy-lactation), потребление кормов (кг/день) и остатки; эффективность корма (л молока / кг сухого вещества) является одним из главных экономических показателей;

- д) управление оборудованием — отслеживание состояния доильных роботов, аномалий работы и прогнозирование технического обслуживания.

Роботизированные доильные установки, такие как Lely Astronaut, автоматически собирают данные о каждом доении. Датчики активности (Lely Qwes) фиксируют шаги, жвачку и время приёма корма. Эти данные передаются в облачный сервис Lely Horizon через REST API. Совокупный объём данных для стада из 500 коров составляет до 250 000 записей в сутки, что делает невозможной их ручную обработку.

Раннее выявление таких заболеваний, как мастит и кетоз, критически важно для экономической эффективности. По данным международного исследования [3], средняя стоимость случая клинического мастита составляет от 100 до 300 долларов США с учётом снижения надоев (70 процентов потерь), затрат на лечение (15 процентов) и преждевременной выбраковки (15 процентов). Кетоз поражает до 40 процентов коров в ранний послеродовой период и при невыявлении приводит к потере до 500 кг молока за лактацию. Модели машинного обучения способны анализировать паттерны в многомерных временных рядах и обнаруживать отклонения на ранних стадиях, когда клинические признаки ещё не очевидны [3].

1.2 Обзор существующих решений

Рассмотренная выше концепция точного животноводства определяет требования к информационным системам, которые должны обеспечивать автоматизированную обработку данных. Далее проведём сравнительный анализ существующих на рынке решений.

На рынке представлены несколько информационных систем управления молочными фермами. Сравнительный анализ приведён в таблице 1.

Таблица 1 – Сравнительный анализ существующих решений

Критерий	Lely Horizon	DeLaval DelPro	SCR Heatime	1С: Управление фермой
Учёт животных	Да	Да	Нет	Да
Учёт надоев	Да	Да	Нет	Да
Воспроизводство	Да	Да	Да	Да
Кормление	Да	Да	Нет	Ограничено
Интеграция с роботами	Lely	DeLaval	Нет	Нет
Модели ML	Нет	Нет	Да (охота)	Нет
Открытый API	Ограничен	Нет	Нет	Нет
Кастомизация	Низкая	Низкая	Низкая	Средняя
Стоимость	Высокая	Высокая	Средняя	Низкая

Lely Horizon — облачная платформа от производителя доильных роботов Lely. Обеспечивает мониторинг стада и оборудования в реальном времени: отображение текущего надоя, активности, жвачки и состояния здоровья по каждой корове. Интерфейс поддерживает настройку пороговых оповещений (например, падение надоя > 20 процентов за сутки). Однако система ограничена экосистемой Lely: не поддерживает оборудование других производителей, не предоставляет API для интеграции сторонних моделей аналитики и не позволяет добавлять собственные правила оповещений. Для ферм, использующих смешанное оборудование, это является критическим ограничением.

DeLaval DelPro — система управления фермой от DeLaval. Предлагает учёт животных и надоев, управление доильным залом и мониторинг оборудования. Является проприетарной и привязана к оборудованию DeLaval: данные доильных роботов других производителей недоступны. Не поддерживает интеграцию с оборудованием сторонних производителей, что затрудняет внедрение на фермах с гетерогенным оборудованием. Предиктивная аналитика ограничена базовыми пороговыми оповещениями.

SCR Heatime — система мониторинга активности и обнаружения охоты от SCR Engineers (ныне Allflex). Система специализируется на воспроизводстве: использует данные о активности (шаги) и жвачке для

обнаружения охоты с помощью запатентованного алгоритма. Встроенная модель машинного обучения обеспечивает чувствительность обнаружения охоты до 95 процентов при специфичности 90 процентов [1]. Однако SCR Heatime не охватывает полный цикл управления фермой: нет учёта кормления, надоев, запасов и оборудования. Система является закрытой и не предоставляет API.

1С: Управление фермой — отечественное решение на платформе 1С:Предприятие. Обеспечивает базовый учёт животных, надоев, воспроизводства и кормления. Преимуществом является локализация, соответствие российскому законодательству и низкая стоимость. Однако система не поддерживает интеграцию с роботизированным оборудованием, не имеет модулей машинного обучения, ограничена в кастомизации отчётов и использует монолитную архитектуру, затрудняющую масштабирование.

Анализ показывает, что ни одно из существующих решений не обеспечивает одновременно:

- а) полный цикл управления стадом от учёта до аналитики;
- б) интеграцию с оборудованием сторонних производителей через открытое API;
- в) встроенные модели машинного обучения для предиктивной аналитики;
- г) открытую расширяемую архитектуру для добавления собственных моделей.

1.3 Техническое задание

Сравнительный анализ показывает, что ни одно из существующих решений не обеспечивает одновременно полный цикл управления стадом, интеграцию с оборудованием сторонних производителей, встроенные модели машинного обучения и открытую архитектуру. На основании этого вывода сформулируем техническое задание на разработку информационной системы.

Учёт животных должен обеспечивать ведение реестра с полной биографической информацией (инвентарный номер, кличка, пол, дата рождения, порода, происхождение), историю переводов между группами и местоположениями, деактивацию без удаления записей для сохранения истории, поиск по кличке и инвентарному номеру с поддержкой нечёткого поиска (триграммное сходство `pg_trgm`), импорт из CSV и экспорт карточки в

PDF, а также пакетные операции для массовой деактивации и обновления до 500 животных.

Учёт надоев молока должен включать суточные надои (объём в кг, жир, белок, лактозу, SCC, FPR), визиты на доение (время начала и окончания, объём по четвертям вымени, скорость доения, данные робота), результаты лабораторных анализов качества молока и мониторинг резервуара-охладителя.

Управление воспроизводством должно поддерживать регистрацию отёлов с записью данных о телёнке, осеменений с указанием быка-производителя, контроль стельности, регистрацию охот на основе данных активности и управление запусками (сухостойный период).

Управление кормлением должно обеспечивать работу с типами кормов и группами кормов, дневными объёмами (плановое и фактическое потребление) и визитами на кормление (время, объём, остатки).

Фитнес-мониторинг и пастьба должны охватывать активность (шаги за период), жвачку (минуты жевания за период) и данные о пастьбе.

Отчёты и аналитика должны включать генерацию отчётов более 17 типов с экспортом в CSV и PDF, ключевые показатели эффективности (KPI), тренды надоев с прогнозом и прогноз стельных коров.

Интеграция с роботизированным оборудованием должна обеспечивать автоматическую синхронизацию с облачным API Lely Horizon с настраиваемым интервалом для 23 типов данных и шифрованием учётных данных (AES-256-GCM).

Предиктивная аналитика должна включать:

- а) прогноз надоев на корову (с доверительными интервалами q10/q90) и на стадо;
- б) обнаружение мастита и кетоза;
- в) выявление охоты;
- г) оценку риска выбраковки;
- д) кластеризацию стада (KMeans);
- е) обнаружение аномалий оборудования (Isolation Forest);
- ж) рекомендации по кормлению;
- и) индекс здоровья на основе Z-оценок;
- к) сравнение моделей прогнозирования (8 моделей с автоматическим выбором).

Нефункциональные требования включают:

- а) аутентификацию и авторизацию с разграничением ролей (admin/user);
- б) адаптивный веб-интерфейс для настольных и мобильных устройств;
- в) контейнеризацию всех компонентов (Docker);
- г) мониторинг работоспособности (Prometheus, Grafana, Loki, Tempo);
- д) автоматизированное развёртывание через CI/CD (GitHub Actions).

1.4 Стек технологий

Сформулированное техническое задание определяет требования к технологическому стеку. Выбор технологий обоснован требованиями к производительности, безопасности и удобству разработки.

Для серверной части выбран язык программирования Rust [4], обеспечивающий безопасность работы с памятью без сборщика мусора за счёт системы владения (ownership) и заимствования (borrowing), что исключает целый класс уязвимостей: использование после освобождения (use-after-free), двойное освобождение (double-free), выход за границы массива. По результатам бенчмарков TechEmpower Framework Benchmarks, веб-фреймворк Actix-web (аналогичного уровня с Axum) демонстрирует пропускную способность более 300 000 запросов/сек на одном ядре, что в 3–5 раз выше Node.js/Express и сопоставимо с C++. Веб-фреймворк Axum [5] построен на асинхронном рантайме Tokio [6] и библиотеке Tower [7], что обеспечивает высокую пропускную способность при обработке HTTP-запросов за счёт неблокирующего ввода-вывода. Библиотека SQLx [8] выполняет проверку SQL-запросов при компиляции (compile-time query checking), исключая ошибки в SQL-коде на этапе сборки.

В качестве основной СУБД выбран PostgreSQL [9] — зрелая (более 25 лет разработки), надёжная (ACID-транзакции) реляционная база данных с расширенной поддержкой типов данных, индексов и хранимых процедур. Расширение TimescaleDB [10] обеспечивает эффективное хранение и агрегацию временных рядов с использованием гипертаблиц [11] и непрерывных агрегатов, что критично для таблиц с суточными надоями, активностью и жвачкой (миллионы записей за год).

Для клиентского приложения выбран фреймворк SvelteKit [12]. В отличие от React и Angular, Svelte 5 [13] не использует виртуальный DOM, а

компилирует компоненты в оптимизированный императивный JavaScript-код: размер бандла в 2–3 раза меньше React-аналога, а время рендеринга — в 1,5–2 раза быстрее. SvelteKit поддерживает серверный рендеринг (SSR), улучшающий скорость начальной загрузки. Tailwind CSS [14] используется для стилизации (utility-first подход с автоматическим удалением неиспользуемых классов), Chart.js [15] — для визуализации данных.

Для модуля машинного обучения выбран Python как основной язык в области data science с наибольшей экосистемой библиотек. FastAPI [16] обеспечивает высокопроизводительный API с автоматической генерацией документации (OpenAPI/Swagger) и валидацией типов через Pydantic. XGBoost [17] используется для задач классификации и регрессии благодаря высокой производительности (в 10–20 раз быстрее scikit-learn на тех же данных) и поддержке квантильной регрессии. Prophet [18] — для прогнозирования на уровне стада с автоматической декомпозицией тренда и сезонности. Optuna [19] — для байесовской оптимизации гиперпараметров (TPE-сэмплер).

Инфраструктура базируется на Docker [20] и Docker Compose [21], обеспечивающих контейнеризацию и оркестрацию с изолированными средами для каждого компонента. Nginx [22] выступает в роли обратного прокси с терминацией SSL, rate limiting и кэшированием статики. Стек мониторинга Grafana LGTM (Prometheus [23], Grafana [24], Loki [25], Tempo [26]) обеспечивает комплексное наблюдение: метрики, логи и распределённые трассировки. GitHub Actions [27] автоматизирует CI/CD: параллельное тестирование трёх компонентов, сборку Docker-образов и деплой.

2 Проектирование и разработка системы управления молочным животноводством

2.1 Архитектура системы

Разработанная информационная система имеет трёхзвенную архитектуру, состоящую из серверной части (backend), клиентского веб-приложения (frontend) и модуля машинного обучения (analytics-ml). Компоненты развёрнуты в контейнерах Docker и объединены через обратный прокси Nginx.

На рисунке 1 представлена архитектура системы. Nginx является единой точкой входа, принимающей HTTP-запросы от клиентов. Запросы к статическим ресурсам и страницам маршрутизируются к контейнеру frontend (SvelteKit), запросы к API — к контейнеру backend (Rust/Axum). Backend обращается к PostgreSQL для хранения данных, к Redis для кэширования и rate limiting, и к analytics-ml для получения прогнозов машинного обучения. Синхронизация с облачным API Lely выполняется периодически по таймеру внутри backend.

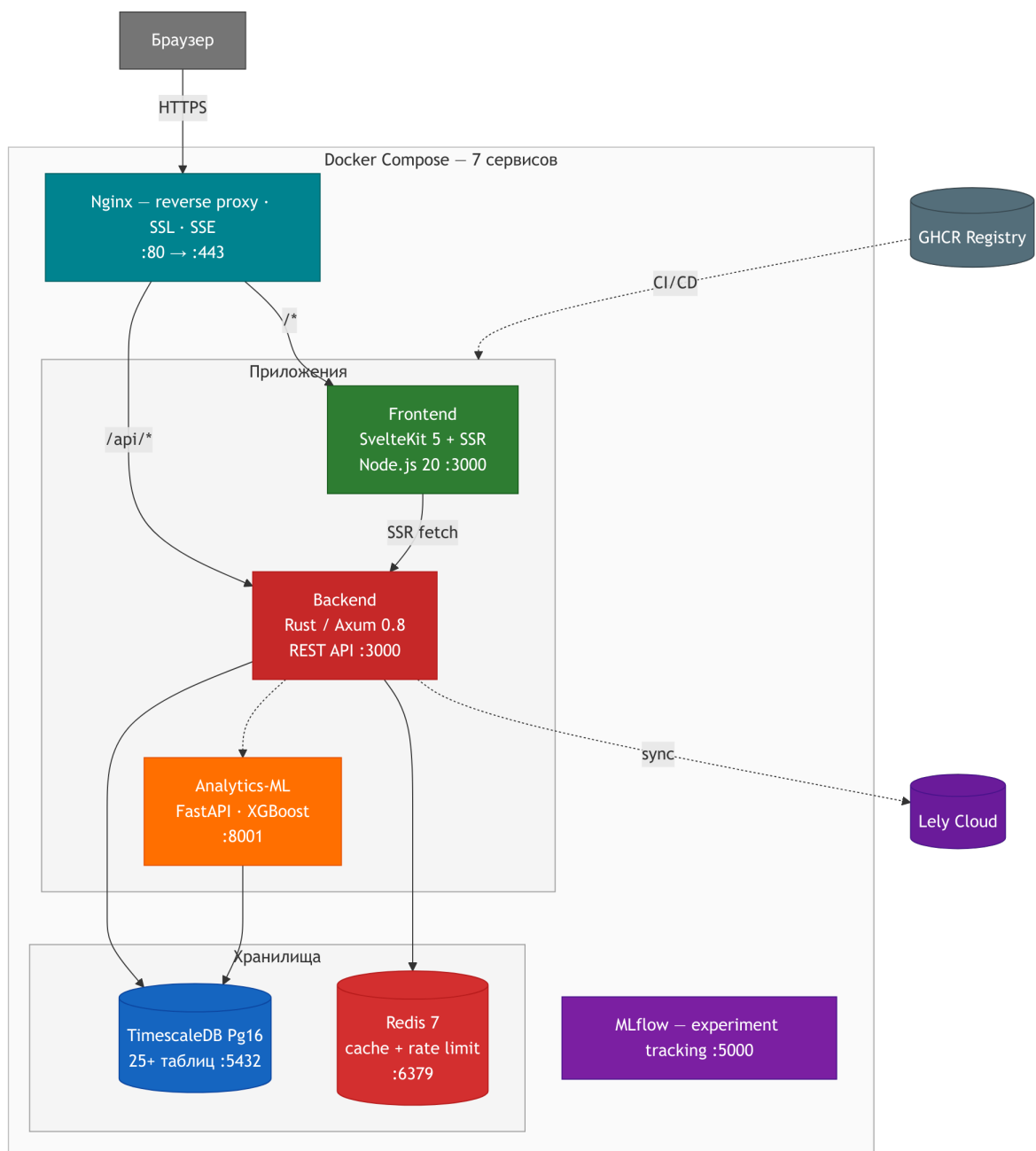


Рисунок 1 – Архитектура информационной системы

Архитектурные решения обоснованы несколькими соображениями. Разделение ответственности обеспечивает чёткое разграничение: backend отвечает за бизнес-логику, frontend — за пользовательский интерфейс, analytics-ml — за модели машинного обучения. Независимое масштабирование позволяет запускать несколько экземпляров backend за балансировщиком нагрузки без изменения остальных компонентов. Технологическая гетерогенность даёт возможность использовать наиболее подходящий стек для каждого компонента (Rust для серверной части, Svelte для клиента, Python для ML). Изоляция сбоев, обеспечиваемая контейнерной

изоляция Docker, гарантирует, что сбой одного компонента не приводит к отказу всей системы.

2.2 Модель данных

Архитектура системы определяет структуру хранилища данных, к описанию которой мы переходим. База данных PostgreSQL содержит более 35 таблиц, обеспечивающих хранение всех сущностей предметной области. Перечень таблиц с назначением приведён в приложении , полная ER-модель — в приложении . На рисунке 2 представлена модель данных системы.

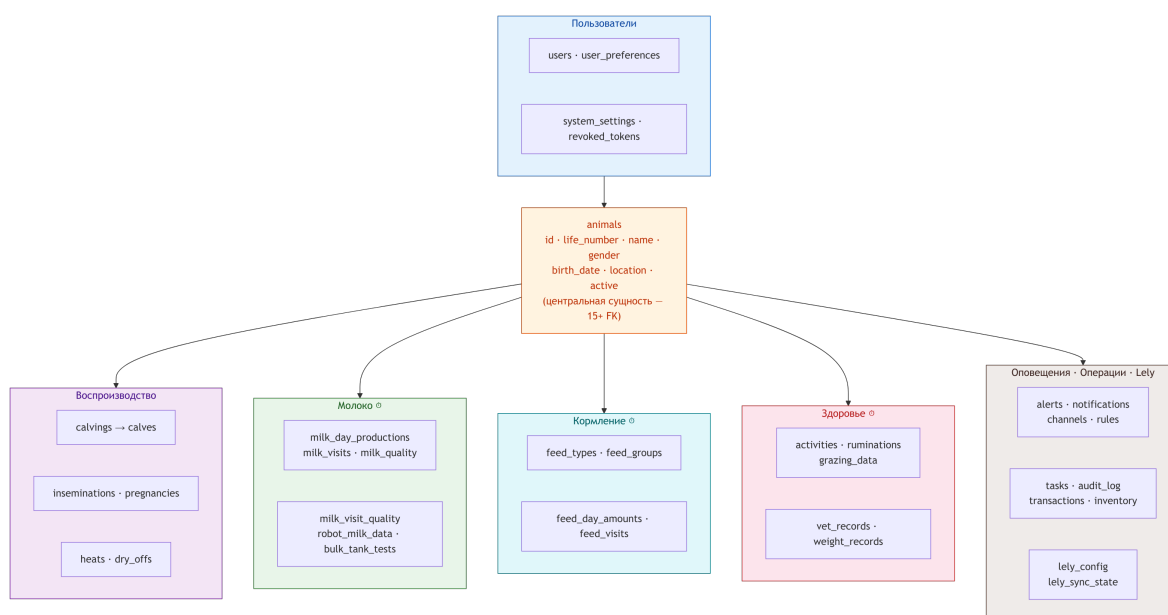


Рисунок 2 – Модель данных информационной системы

Основные таблицы базы данных и их назначение:

users — пользователи системы. Содержит имя пользователя, хеш пароля (bcrypt), роль (admin/user), флаг обязательной смены пароля и дату создания. Пароли хранятся в виде хешей с использованием алгоритма bcrypt с автоматической генерацией соли, что исключает утечку паролей при компрометации базы данных.

animals — реестр животных. Содержит уникальный идентификатор, инвентарный номер (life_number), кличку, пол, дату рождения, породу, местоположение, статус активности и множество других полей. Поддерживается флаг active для деактивации животных без удаления записей, что обеспечивает сохранение истории.

milk_day_productions — суточные надои. Привязаны к животному и

дате. Содержат объём молока (кг), жир (%), белок (%), лактозу (%), количество соматических клеток (SCC), энергию (MJ) и жирowo-белковое соотношение (FPR). Композитный индекс (animal_id, date) обеспечивает быстрый поиск надоев за период.

milk_visits — визиты на доение. Содержат информацию о каждом доении: время начала и окончания, объём молока, скорость доения (кг/мин), данные по четвертям вымени (объём, время, процент выдаивания), флаг успешности доения и идентификатор робота.

milk_quality — результаты лабораторных анализов молока. Включает количество соматических клеток (SCC), жир, белок, лактозу, мочевину, бактерии.

calvings, calves, inseminations, pregnancies, heats, dry_offs — данные о воспроизводстве. Каждая таблица связана с животным через внешний ключ. Поддерживается полная история всех событий воспроизводства, включая дату события, результат и примечания.

feed_types, feed_groups, feed_day_amounts, feed_visits — данные о кормлении. Типы кормов описывают рационы с указанием стоимости и единиц измерения, группы — привязку кормов к группам животных, дневные объёмы — плановое и фактическое потребление, визиты — отдельные посещения кормостанции с временем и объёмом.

activities, ruminations, grazing_data — данные фитнес-мониторинга и пастьбы. Активность измеряется в шагах за период, жвачка — в минутах жевания за период.

alerts — система оповещений. Оповещения создаются автоматически на основе настраиваемых пороговых значений. Каждое оповещение содержит категорию (milk, health, reproduction, feed, equipment), степень важности (high, medium, low), статус (active, acknowledged, resolved), идентификатор животного и текстовое описание.

lely_sync_state — состояние синхронизации с Lely. Хранит дату последней успешной синхронизации для каждого типа данных, что позволяет инкрементально обновлять информацию и минимизировать объём данных при каждой синхронизации.

lely_config — конфигурация интеграции с Lely. Хранит URL API, зашифрованные логин и пароль, ключ фермы и настройки синхронизации. Учётные данные шифруются с использованием AES-256-GCM [28].

Для эффективной работы с временными рядами (надои, активность, жвачка) используется расширение TimescaleDB [10]. Таблицы с временными рядами преобразованы в гипертаблицы с автоматической партицировкой по времени (по умолчанию — 7 дней на чанк), что ускоряет запросы за определённый период, так как сканируются только релевантные чанки. Непрерывные агрегаты (continuous aggregates) обеспечивают предвычисление суточных и часовых агрегатов, обновляемых в фоновом режиме.

Индексация включает уникальные ограничения (инвентарный номер животного), композитные индексы (животное + дата) и частичные индексы (только активные животные). Расширение pg_trgm [29] обеспечивает полнотекстовый поиск по кличкам животных с использованием триграммного сходства.

2.3 Серверная часть (Backend)

Спроектированная модель данных служит основой для серверной части, обеспечивающей бизнес-логику и API для клиентского приложения. Серверная часть реализована на языке Rust с использованием фреймворка Axum. Архитектура следует паттерну разделения на слои: HTTP-обработчики (handlers), сервисы бизнес-логики (services), модели данных (models) и промежуточное программное обеспечение (middleware). На рисунке 3 представлена архитектура серверной части.

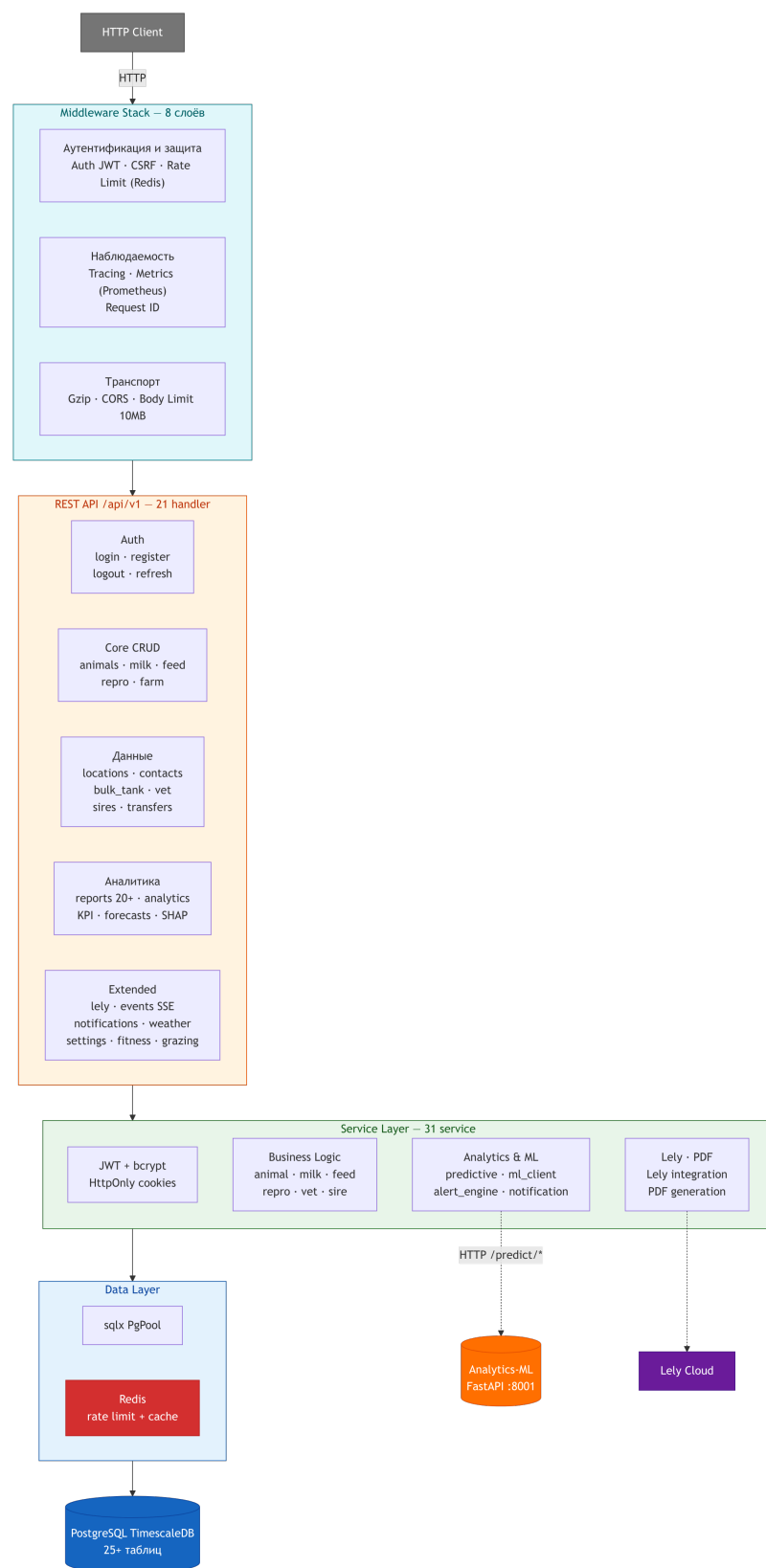


Рисунок 3 – Архитектура серверной части

2.3.1 Маршрутизация и обработка запросов

Все API-эндпоинты доступны по префиксу `/api/v1/`. Маршруты

сгруппированы по доменным областям: животные, надои, воспроизводство, кормление, фитнес, отчёты, аналитика, настройки и другие. Общее количество обработчиков — более 25 модулей. Примеры запросов и ответов приведены в приложении .

Документация API генерируется автоматически с помощью библиотеки `utoipa` [30] на основе аннотаций в коде и доступна через Swagger UI по адресу `/api/v1/docs`. Аннотации включают описание эндпоинтов, типы запросов и ответов, коды ошибок.

2.3.2 Аутентификация и авторизация

Аутентификация основана на JWT [31] (JSON Web Token) с алгоритмом HS256. При входе в систему сервер выдаёт два токена: короткоживущий `access token` (15 минут) и долгоживущий `refresh token` (7 дней) для обновления `access token` без повторного ввода пароля.

Токены хранятся в `HttpOnly, SameSite=Strict cookie`, что предотвращает доступ к ним из JavaScript и обеспечивает защиту от CSRF-атак [32]. Отзыв токенов реализован через таблицу `revoked_tokens`, хранящую JTI отозванных токенов. Просроченные записи удаляются ежечасно фоновым заданием.

Авторизация реализована через три уровня доступа, соответствующих типам Ахит-экстракторов. Уровень **Claims** допускает любого аутентифицированного пользователя с валидным `access token`. Уровень **ClaimsAllowMustChange** допускает состояние обязательной смены пароля. Уровень **AdminGuard** ограничивает доступ пользователями с ролью `admin`.

2.3.3 Промежуточное программное обеспечение (Middleware)

Стек `middleware` обеспечивает безопасность, наблюдаемость и ограничение нагрузки. **CORS** настраивает кросс-доменный доступ [33] с конфигурируемыми источниками. **CSRF-защита** проверяет заголовок `Origin` для изменяющих состояние запросов. **Rate Limiting** ограничивает частоту запросов по IP-адресу (100 запросов за 60 секунд) с использованием Redis. **Gzip-сжатие** уменьшает объём передаваемых данных. **Логирование** обеспечивает трассировку HTTP-запросов через `tracing` с поддержкой структурированных логов в JSON-формате. **Метрики Prometheus** включают

счётчики `http_requests_total` и гистограммы `http_request_duration_seconds` с нормализацией маршрутов.

2.3.4 Интеграция с Lely

Модуль интеграции с Lely обеспечивает автоматическую двустороннюю синхронизацию данных с облачным API Lely Horizon. Синхронизация выполняется по настраиваемому таймеру (по умолчанию каждые 5 минут).

Архитектура модуля включает HTTP-клиент с аутентификацией по токену и автоматическим обновлением каждые 58 минут, а также повторными попытками с экспоненциальной задержкой. Планировщик синхронизации поддерживает параллельные задачи и инкрементальную синхронизацию по временным диапазонам (чанками по 7 дней). Маппинг данных использует кэш инвентарных номеров для разрешения Life Number Lely во внутренний идентификатор животного. Upsert-логика выполняет добавление или обновление записей через `INSERT ON CONFLICT UPDATE`. Учётные данные шифруются алгоритмом AES-256-GCM [28].

Синхронизируются 23 типа данных: животные, отёлы, осеменения, стельности, охоты, запуски, суточные надои, визиты на доение, качество молока, данные роботов, кормление, активности, жвачка, пастьба, быки-производители, переводы, кровные линии, типы и группы кормов, контакты, местоположения.

2.4 Клиентское веб-приложение (Frontend)

Серверный API, описанный выше, используется клиентским веб-приложением, обеспечивающим пользовательский интерфейс системы. Клиентское приложение реализовано на SvelteKit 2 с Svelte 5 [13] и представляет собой одностраничное приложение (SPA) с серверным рендерингом (SSR). На рисунке 4 представлена архитектура клиентского приложения.

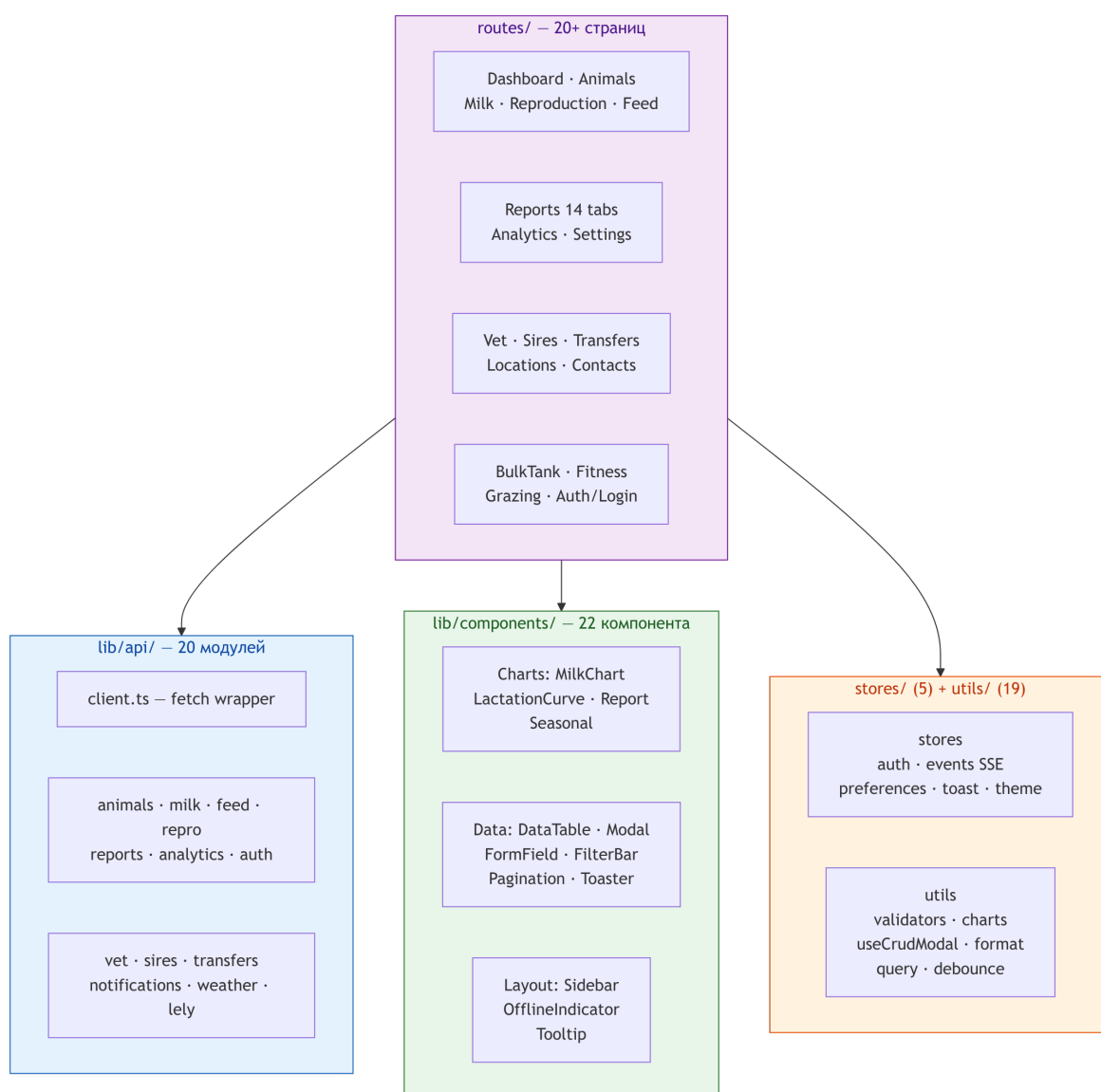


Рисунок 4 – Архитектура клиентского веб-приложения

2.4.1 Маршрутизация и навигация

Приложение содержит более 24 страниц, сгруппированных по функциональным областям: дашборд, животные, надой, воспроизводство,

кормление, фитнес, пастьба, отчёты, аналитика, настройки, контакты, местоположения, быки, переводы, запасы, оповещения, задачи, ветеринария, финансы, рентабельность, поиск.

Навигация реализована через боковую панель (Sidebar) с иконками из библиотеки Lucide и группировкой разделов. Адаптивная верстка на основе Tailwind CSS обеспечивает корректное отображение на мобильных устройствах: на экранах менее 768px боковая панель сворачивается в гамбургер-меню.

2.4.2 Аутентификация на стороне клиента

Серверный хук (`hooks.server.ts`) проверяет наличие JWT в cookie при каждом запросе. Декодируется полезная нагрузка токена и проверяется срок действия. Неаутентифицированные пользователи перенаправляются на страницу входа, аутентифицированные — не могут перейти на страницу входа.

Управление состоянием аутентификации реализовано через Svelte-стор (`auth store`), который инициализируется при загрузке приложения через `endpoint refresh`.

2.4.3 API-клиент

Клиент для взаимодействия с серверным API реализует автоматическое обновление токена: при получении ответа 401 выполняется попытка обновления access token через `/auth/refresh`; если refresh-токен также недействителен, пользователь разлогинивается и перенаправляется на страницу входа.

Для повышения надёжности реализован механизм повторных попыток: до 3 попыток с экспоненциальной задержкой (500 мс базовая задержка + случайный джиттер) для ответов с кодом 5xx. API-клиент разделён на 23 модуля по доменным областям.

2.4.4 Компоненты пользовательского интерфейса

Библиотека переиспользуемых компонентов включает DataTable — таблицу данных с сортировкой по столбцам, серверной пагинацией и адаптивным отображением на мобильных устройствах; Modal и

ConfirmDialog — модальные окна для создания, редактирования и подтверждения действий; FormField — компонент формы с поддержкой валидации и отображения ошибок; FilterBar — панель фильтров с поддержкой текстового поиска, выпадающих списков и диапазонов дат; MilkChart, LactationCurveChart, ReportChart, SeasonalChart — компоненты визуализации данных на основе Chart.js; Toaster — уведомления с раскрываемыми подробностями и автоматическим скрыванием.

Валидация форм реализована на основе Svelte 5 Runes — реактивной системы состояния, позволяющей декларативно описывать правила валидации и отслеживать состояние формы (dirty, errors, touched).

2.4.5 Генерация отчётов

Система поддерживает генерацию отчётов более 17 типов, покрывающих все аспекты управления стадом. Каждый отчёт доступен в формате JSON для отображения в интерфейсе, а также поддерживает экспорт в CSV и PDF. Ключевые типы отчётов перечислены в таблице 2.

Таблица 2 – Основные типы отчётов системы

Код	Название	Описание
R12	Здоровье вымени	Рабочий список коров с отклонениями за 24 часа
R13	Неудачные доения	Визиты с неуспешным доением
R16	Обзор стада	Дневная производительность стада
R18	Остаток корма	Невыданный корм по коровам
R20	Надой по времени	Суточный надой с разбивкой по показателям
R23	Анализ вымени	Расширенный анализ за 14 дней
R24	Активность/жвачка	Здоровье, активность и жвачка
R31-34	Календарь	Ожидаемые отёлы, запуски, охота, стельность
R41	Эффективность	Эффективность корова-робот
R52	Лактация	Кривые лактации по номерам
R56	Робот	Производительность роботов
R70	Корм по типам	Стоимость и объём кормов по типам
R72	Корм на корову	Потребление корма на корову

2.5 Модуль машинного обучения (Analytics-ML)

Наряду с серверной частью и клиентским приложением, система включает модуль машинного обучения, отвечающий за предиктивную аналитику. Модуль реализован на Python с использованием FastAPI и предоставляет API для прогнозирования и обнаружения аномалий. На рисунке 5 представлена архитектура модуля машинного обучения.

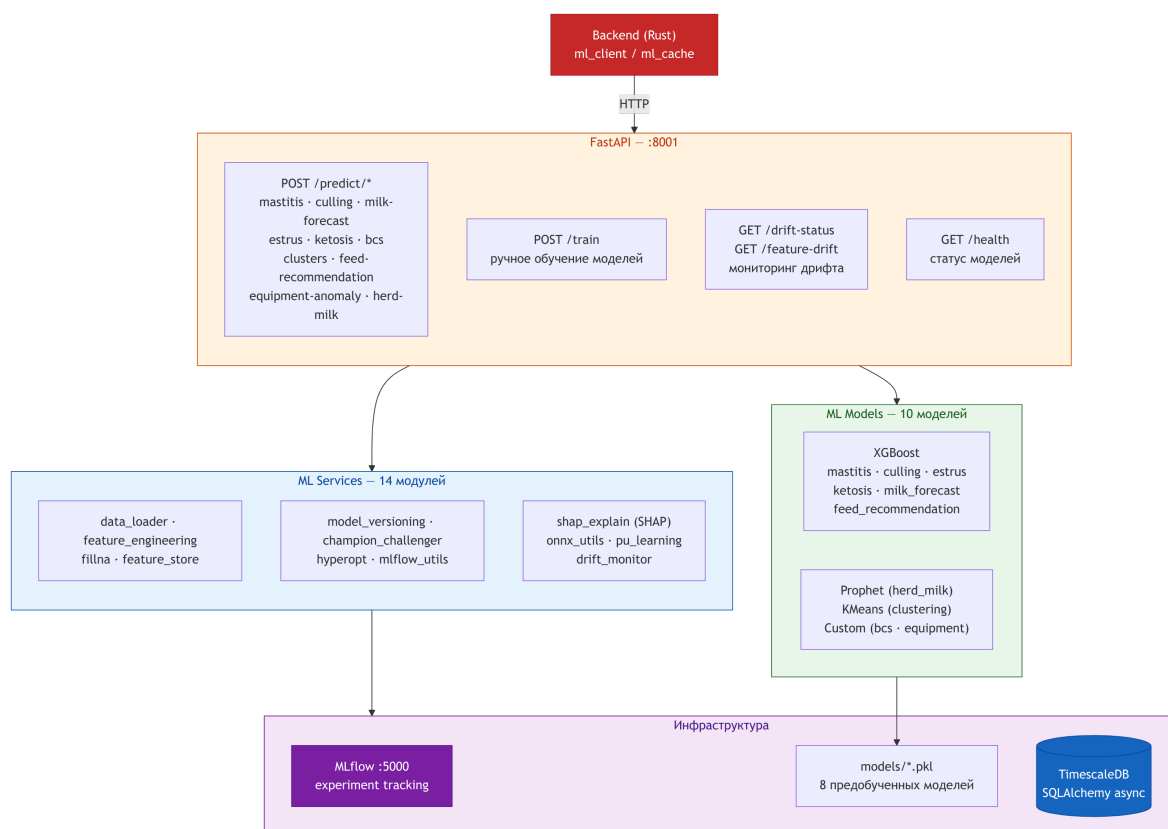


Рисунок 5 – Архитектура модуля машинного обучения

2.5.1 Выбор алгоритмов

Все задачи предиктивной аналитики в системе формулируются на табулярных данных, образованных агрегированными показателями животных (надой, активность, жвачка, SCC) за скользящие временные окна. Для таких данных методы градиентного бустинга показывают состояние-of-the-art результаты [17], превосходя нейросетевые подходы на выборках умеренного размера (сотни—тысячи записей), характерных для одной фермы.

Система автоматически выбирает наилучший алгоритм градиентного бустинга среди трёх реализаций: XGBoost [17], LightGBM [34] и CatBoost [35]. Выбор обусловлен различиями в подходах к построению

деревьев: XGBoost использует уровень-wise рост, обеспечивая стабильность на сбалансированных данных; LightGBM применяет лист-wise рост с гистограммной оптимизацией, что ускоряет обучение на больших выборках; CatBoost использует ordered boosting, устойчивый к утечке целевой переменной (target leakage). Гипероптимизация Optuna проводит испытания равномерно по всем трём бэкендам и выбирает лучший по результатам кросс-валидации.

Для кластеризации стада используется алгоритм KMeans [36], либо HDBSCAN [37] при наличии кластеров произвольной формы; выбор производится автоматически по метрике силуэта. Для обнаружения аномалий оборудования применяется Isolation Forest [38], не требующий размеченных данных и устойчивый к высокой доле аномалий. Для прогнозирования на уровне стада используется Prophet [18], автоматически декомпозирующий временной ряд на тренд, сезонность и эффекты особых дней.

Разработано 11 моделей машинного обучения (таблица 3).

Таблица 3 – Модели машинного обучения

Модель	Алгоритм	Метрика	Назначение
Прогноз надоев (корова)	XGBoost, квантильная регрессия	MAE	Дневной надой с q10/q90 интервалами
Прогноз надоев (стадо)	Prophet	—	Тренд надоя стада с сезонностью
Риск мастита	XGBoost / LightGBM / CatBoost	ROC-AUC	Вероятность мастита
Предупреждение кетоза	XGBoost / LightGBM / CatBoost	ROC-AUC	Риск метаболического нарушения
Обнаружение охоты	XGBoost / LightGBM / CatBoost	ROC-AUC	Состояние (в охоте / нет)
Риск выбраковки	XGBoost / LightGBM / CatBoost	R^2	Дни до выбраковки
Оценка BCS	XGBoost / LightGBM / CatBoost	MAE	Упитанность (1,0–5,0)
Кластеризация стада	KMeans / HDBSCAN	Silhouette	Сегментация стада
Аномалии оборудования	Isolation Forest	Доля аномалий	Аномалии работы роботов
Рекомендации по корму	XGBoost / LightGBM / CatBoost	MAE	Объем корма
Комплексное здоровье	MultiOutput Classifier	ROC-AUC	Мастит + кетоз + охота совместно

2.5.2 Обоснование выбора метрик

Для задач классификации (мастит, кетоз, охота) выбрана метрика ROC-AUC (Area Under the Receiver Operating Characteristic Curve), поскольку она инвариантна к порогу принятия решения и соотношению классов. Это

критично для veterinary-задач, где положительный класс составляет менее 5 процентов наблюдений: метрики accuracy и precision при таком дисбалансе неинформативны. ROC-AUC оценивает способность модели ранжировать наблюдения по степени риска, что соответствует прикладной задаче — выделить животных с наибольшей вероятностью заболевания для приоритетного осмотра.

Для задач регрессии (прогноз надоев, рекомендации по корму, BCS) выбрана MAE (Mean Absolute Error), а не MSE/ RMSE. Обоснование: MAE менее чувствительна к выбросам, которые характерны для данных доильных роботов (пропущенное доение, сбой датчика). Кроме того, MAE измеряется в тех же единицах, что и целевая переменная (кг молока, кг корма), что упрощает интерпретацию результатов фермером.

Для кластеризации выбрана метрика силуэта, оценивающая компактность и разделимость кластеров без необходимости внешних меток.

Оценка всех моделей производится методом k -кратной кросс-валидации, где число фолдов адаптируется к объёму данных: $k = \min(5, \max(2, N/n))$, где N — количество наблюдений, n — минимальный размер фолда.

2.5.3 Формирование обучающих меток

Особенностью задач обнаружения заболеваний является отсутствие надёжных размеченных данных: ветеринарные записи неполны, а диагноз часто ставится с задержкой. Для решения этой проблемы применён подход обучения на положительных и неразмеченных данных (PU-learning) [39].

Метка «мастит» формируется на основе комбинации клинически обоснованных правил: $SCC > 300\,000$ клеток/мл; либо $SCC > 200\,000$ с трендом роста $> 1,5$; либо $SCC > 150\,000$ при повышенной электропроводности > 55 мСм/см; либо отклонение надоя > 15 процентов при $SCC > 100\,000$. Аналогичные правила на основе FPR и жвачки используются для кетоза, а на основе соотношения активности и жвачки — для охоты.

При наличии подтверждённых ветеринарных диагнозов они замещают эвристические метки с увеличением веса наблюдения в 3 раза, что позволяет модели постепенно переходить от semi-supervised к fully-supervised режиму по мере накопления данных.

2.5.4 Прогноз надоев с доверительными интервалами

Для прогнозирования надоев на корову обучаются три модели квантильной регрессии: для медианы (q50), нижней границы (q10) и верхней границы (q90). Объективная функция Quantile Loss для квантиля α :

$$L_{\alpha}(y, \hat{y}) = \begin{cases} \alpha \cdot (y - \hat{y}), & \text{если } y \geq \hat{y} \\ (1 - \alpha) \cdot (\hat{y} - y), & \text{если } y < \hat{y} \end{cases}$$

где y — фактическое значение, $\hat{y}$ — прогноз. Интервал q10–q90 содержит 80 процентов распределения прогнозов, позволяя фермеру количественно оценивать неопределённость.

2.5.5 Инженерия признаков

Для каждой модели формируется расширенный набор признаков на основе данных из PostgreSQL с использованием LATERAL JOIN для вычисления скользящих агрегатов. Признаки включают:

- а) лаговые — значения целевого показателя за предыдущие 1, 7, 14, 30 дней;
- б) скользящие статистики — среднее, стандартное отклонение, коэффициент вариации за окна 7, 14, 30 дней;
- в) трендовые — отношение recent/baseline, наклон линейной регрессии за 7 и 14 дней;
- г) производные — FPR, соотношение надой/корм, соотношение активность/жвачка;
- д) физиологические — DIM, номер лактации, возраст;
- е) метеорологические — температура, влажность, индекс температурно-влажностного стресса (THI);
- ж) исторические — количество ветеринарных обработок за 180 дней, дней с последней обработки;
- и) временные ряды — энтропия, автокорреляция на лаге 1 и 7, сила тренда и сезонности.

Для модели прогноза надоев дополнительно вычисляются TSFRESH-подобные признаки: приближённая энтропия, коэффициент эксцесса, максимальная просадка (drawdown), сила сезонности по

STL-декомпозиции. Эти признаки позволяют модели различать стабильных и нестабильных коров.

Обработка пропущенных значений выполняется медианой по стаду, вычисленной на этапе обучения и сохраняемой вместе с моделью для обеспечения консистентности при прогнозировании. Дополнительно выполняется отбор признаков с помощью RandomForest importance: отбираются признаки с важностью выше медианной.

2.5.6 MLOps: оптимизация, мониторинг дрейфа и управление моделями

Оптимизация гиперпараметров осуществляется библиотекой Optuna [19] с TPE-сэмплером (Tree-structured Parzen Estimator) в 8-мерном пространстве (learning_rate, max_depth, n_estimators, min_child_weight, subsample, colsample_bytree, reg_alpha, reg_lambda). Каждое испытание оценивается кросс-валидацией; по умолчанию выполняется 30 испытаний с таймаутом 120 секунд.

Мониторинг дрейфа реализован тремя методами. **Prediction drift** — Z-тест: если $z = |\bar{x}_{\text{recent}} - \bar{x}_{\text{baseline}}| / \sigma_{\text{baseline}} > 2,0$, фиксируется сдвиг распределения прогнозов. **Feature drift** — для каждого признака вычисляется Z-тест и критерий Колмогорова–Смирнова; дрейф фиксируется при $z > 2,5$ или $p_{KS} < 0,05$. **MMD-drift** — Maximum Mean Discrepancy с RBF-ядром, оценивающий расстояние между распределениями в пространстве признаков; порог $\text{MMD} > 0,1$.

При обнаружении дрейфа модель помечается для скорого переобучения, которое выполняется через APScheduler с настраиваемым расписанием (по умолчанию еженедельно).

Для управления моделями реализован подход «чемпион–претендент»: переобученная модель (претендент) сравнивается с текущей (чемпионом) на тех же данных; модель повышается только при улучшении метрики более чем на 1 процент. Это предотвращает деградацию при переобучении на зашумлённых данных.

Объяснимость прогнозов реализована через SHAP [40] с использованием TreeExplainer для древовидных моделей. Отслеживание экспериментов осуществляется через MLflow [41]. Для ускорения инференса модели экспортируются в формат ONNX.

2.6 Предиктивная аналитика в серверной части

Модуль машинного обучения на Python обеспечивает высокую точность прогнозов, однако его доступность зависит от состояния отдельного сервиса: при отказе или перезапуске analytics-ml предиктивная аналитика становится недоступна. Для обеспечения отказоустойчивости серверная часть содержит собственные алгоритмы аналитики, реализованные на Rust. Эти алгоритмы работают внутри процесса backend и не зависят от внешних сервисов, что гарантирует доступность базовой аналитики в любой момент.

Метод Хольта–Винтерса [42] — тройное экспоненциальное сглаживание для прогнозирования временных рядов надоев. Модель учитывает три компоненты: уровень (level), тренд (trend) и сезонность (seasonality). Рекуррентные формулы:

$$l_t = \alpha \cdot (y_t - s_{t-m}) + (1 - \alpha) \cdot (l_{t-1} + b_{t-1}) \quad (1)$$

$$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1} \quad (2)$$

$$s_t = \gamma \cdot (y_t - l_t) + (1 - \gamma) \cdot s_{t-m} \quad (3)$$

где l_t — уровень, b_t — тренд, s_t — сезонная компонента, m — период сезонности (7 дней), α , β , γ — параметры сглаживания. Прогноз на h шагов вперёд:

$$\hat{y}_{t+h} = l_t + h \cdot b_t + s_{t+h-m}$$

Для обнаружения выбросов используется межквартильный размах (IQR): значение считается выбросом, если оно выходит за пределы $[Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}]$. Для обнаружения структурных сдвигов применяется t-критерий: если среднее последних 7 дней значительно отличается от среднего предыдущих 14 дней ($p\text{-value} < 0.05$), фиксируется структурный сдвиг.

Кривая лактации Вуда [43] — лог-линейная модель, описывающая зависимость надоя от дня лактации:

$$y(t) = a \cdot t^b \cdot e^{-ct}$$

где $y(t)$ — надой в день лактации t , a — масштабный коэффициент, определяющий начальный уровень надоя, b — параметр, контролирующий скорость нарастания до пика лактации, c — параметр, определяющий скорость спада после пика. Логарифмирование приводит к линейному виду:

$$\ln y(t) = \ln a + b \cdot \ln t - c \cdot t$$

Параметры оцениваются методом наименьших квадратов. Модель позволяет определять день пика лактации $t_{\max} = -b/c$ и сравнивать фактические надои с ожидаемыми для выявления отклонений.

Индекс здоровья — составной показатель на основе z-оценок:

$$\text{Health Index} = -\frac{1}{n} \sum_{i=1}^n |z_i|$$

где $n = 4$ — количество показателей, $z_i = (x_i - \mu_i)/\sigma_i$ — стандартизованное отклонение i -го показателя (надой, SCC, активность, жвачка) от среднего μ_i и стандартного отклонения σ_i по стаду. Индекс принимает значения от $-\infty$ до 0, где значения ниже -2 указывают на потенциальные проблемы со здоровьем.

Оптимизатор запуска — определяет оптимальную дату сухостойного периода на основе прогноза лактации и планового отёла. Стандартный период запуска — 60 дней до отёла. Оптимизатор корректирует дату с учётом текущего надоя.

2.6.1 Сравнение моделей прогнозирования временных рядов

Для повышения точности прогнозирования надоев реализована система автоматического сравнения 8 моделей прогнозирования. Исторические данные (90 дней) разбиваются на обучающую (80 процентов) и тестовую (20 процентов) выборки хронологически.

MAPE (Mean Absolute Percentage Error) — основная метрика качества:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$$

где n — количество наблюдений тестовой выборки, y_i — фактическое

значение надоя, $\hat{y}_i$ — прогнозируемое значение. MAPE выбран как основная метрика, поскольку она инвариантна к масштабу (надой разных коров могут отличаться в 2–3 раза).

RMSE (Root Mean Squared Error) дополняет MAPE, показывая типичное отклонение прогноза в абсолютных единицах (кг):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

В отличие от MAPE, RMSE усиливает штраф за крупные ошибки благодаря возведению в квадрат, что позволяет выявлять редкие, но значительные отклонения прогноза.

Лучшей моделью признаётся та, которая демонстрирует минимальный MAPE на тестовой выборке. Реализованы следующие модели:

1. SES (Simple Exponential Smoothing) — простое экспоненциальное сглаживание:

$$\hat{y}_{t+1} = \alpha \cdot y_t + (1 - \alpha) \cdot \hat{y}_t$$

где y_t — фактическое значение надоя в момент t , $\hat{y}_t$ — сглаженное значение, $\alpha \in (0, 1)$ — параметр сглаживания, оптимизируемый перебором по сетке $\{0,05, 0,10, \dots, 0,95\}$ с выбором минимального SSE (суммы квадратов ошибок).

2. SMA (Simple Moving Average) — простое скользящее среднее с окном w .

3. WMA (Weighted Moving Average) — взвешенное скользящее среднее с линейно возрастающими весами:

$$\hat{y}_{t+1} = \frac{\sum_{i=1}^w w_i \cdot y_{t-w+i}}{\sum_{i=1}^w w_i}$$

где w — размер окна (по умолчанию 7 дней), $w_i = i$ — линейно возрастающий вес, придающий большую значимость недавним наблюдениям.

4. Linear Regression — линейная регрессия по времени: $\hat{y}_t = a + b \cdot t$, где a — свободный член, b — наклон тренда, оцениваемые методом наименьших квадратов. Подходит для данных с выраженным линейным

трендом, но не улавливает сезонность.

5. Holt's Linear Method — двойное экспоненциальное сглаживание:

$$l_t = \alpha \cdot y_t + (1 - \alpha) \cdot (l_{t-1} + b_{t-1}) \quad (4)$$

$$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1} \quad (5)$$

где l_t — уровень, b_t — тренд, α и β — параметры сглаживания уровня и тренда соответственно. Прогноз: $\hat{y}_{t+h} = l_t + h \cdot b_t$.

6. Holt-Winters — тройное экспоненциальное сглаживание с сезонной компонентой ($m = 7$ дней).

7. Fourier Series — модель на основе рядов Фурье с линейным трендом:

$$\hat{y}_t = (a + b \cdot t) + \sum_{k=1}^K \left[A_k \cos\left(\frac{2\pi t}{T_k}\right) + B_k \sin\left(\frac{2\pi t}{T_k}\right) \right]$$

где $a + b \cdot t$ — линейный тренд, K — количество гармоник (по умолчанию 3), $T_k = 7/k$ — период k -й гармоники в днях, A_k и B_k — коэффициенты Фурье, оцениваемые методом наименьших квадратов. Базовый период 7 дней соответствует недельной сезонности надоев.

8. ARIMA(0,1,1) — модель авторегрессии интегрированного скользящего среднего:

$$\Delta \hat{y}_t = \mu + \theta \cdot \varepsilon_{t-1}$$

где $\Delta \hat{y}_t = y_t - y_{t-1}$ — первая разность надоя, μ — средний дрейф (среднее суточное приращение), θ — параметр скользящего среднего, ε_{t-1} — ошибка прогноза на предыдущем шаге. Данная спецификация эквивалентна экспоненциальному сглаживанию с дрейфом; для прогноза на $h \geq 2$ шагов вклад МА-члена равен нулю, и прогноз вырождается в прямую с наклоном μ .

2.6.2 Ансамблевый прогноз

Для повышения робастности прогнозирования реализован ансамблевый метод, объединяющий три источника прогноза: ML-модель (XGBoost), нейросетевую модель (N-BEATS [44]) из ML-сервиса и лучшую статистическую модель (Rust). Нейросетевая модель N-BEATS использует

архитектуру с базисными расширениями, способную улавливать нелинейные паттерны во временных рядах, недоступные статистическим методам.

Итоговый прогноз формируется как взвешенное среднее:

$$\hat{y}_t = w_{\text{ML}} \cdot \hat{y}_t^{\text{ML}} + w_{\text{NBEATS}} \cdot \hat{y}_t^{\text{NBEATS}} + w_{\text{Rust}} \cdot \hat{y}_t^{\text{Rust}}$$

Вес каждой модели определяется обратно пропорционально её MAPE: $w_i = (1/(\text{MAPE}_i + 1)) / \sum_j (1/(\text{MAPE}_j + 1))$. MAPE ML-модели и N-BEATS задаётся оценочно по результатам кросс-валидации (8 и 10 процентов соответственно), MAPE Rust-модели вычисляется динамически для каждого животного. Такое взвешивание придаёт больший вес более точным моделям, сохраняя при этом вклад каждой.

Если ML-сервис недоступен, система автоматически откатывается к прогнозу лучшей Rust-модели с весом 1,0, что гарантирует доступность прогноза при любом состоянии инфраструктуры.

2.6.3 Объяснимость прогнозов (SHAP)

Для обеспечения интерпретируемости прогнозов ML-сервис использует метод SHAP [40], основанный на теории кооперативных игр Шепли. Для каждого прогноза вычисляются SHAP-значения, показывающие вклад каждого признака в отклонение от базового значения модели:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

где ϕ_i — SHAP-значение признака i , F — множество всех признаков, S — подмножество признаков, не содержащее i , $f(S)$ — прогноз модели на подмножестве признаков S , а дробь определяет вес каждой коалиции по Шепли. Для XGBoost используется оптимизированный TreeExplainer, вычисляющий точные SHAP-значения за линейное время.

В интерфейсе SHAP-значения визуализируются в виде горизонтальной столбчатой диаграммы с цветовой кодировкой: зелёный для положительного вклада (увеличивает прогноз), красный для отрицательного (уменьшает). Это позволяет фермеру понять, какие именно факторы повлияли на конкретный прогноз.

2.7 Оптимизация производительности

При проектировании системы учитывалось, что объём данных может достигать сотен тысяч животных и десятков миллионов записей. Для обеспечения приемлемого времени отклика при работе с набором данных FelBenitez [45], содержащим более 210 000 животных и 12 600 000 записей надоев, был проведён комплексный анализ узких мест и реализован ряд оптимизаций на всех уровнях архитектуры.

2.7.1 Анализ узких мест

Профилирование запросов к базе данных с помощью EXPLAIN ANALYZE выявило три основные категории проблем:

- а) **Коррелированные подзапросы** — каждый предиктивный эндпоинт выполнял от 10 до 17 LATERAL JOIN на каждое животное. Для 210 000 животных и 15 подзапросов это составляет свыше 3 000 000 executions подзапросов за один HTTP-запрос.
- б) **Повторная загрузка моделей** — функция `joblib.load()` вызывалась при каждом предикте, выполняя чтение с диска и десериализацию объекта XGBoost.
- в) **Отсутствие кеширования** — результаты аналитических запросов и ML-предиктов не кешировались; повторные запросы выполняли полный цикл расчётов.

В таблице 4 приведены типичные времена ответа до оптимизации для ключевых эндпоинтов.

Таблица 4 – Время ответа ключевых эндпоинтов до оптимизации

Эндпоинт	Время ответа, с
Дашборд (/analytics/dashboard)	5–10
Предикт мастита (одно животное)	30–60
Предикт мастита (всё стадо)	180–300
Временная шкала здоровья (90 дней)	10–30
Загрузка модели с диска	0,05–0,2

2.7.2 Оптимизация базы данных

Для ускорения аналитических запросов к TimescaleDB реализован

комплекс мер: композитные индексы, автоматическое сжатие исторических данных и материализованные агрегаты.

Композитные индексы. Большинство коррелированных подзапросов используют паттерн `WHERE animal_id = $1 ORDER BY date DESC LIMIT 1`. Для данного паттерна созданы составные индексы, позволяющие выполнить запрос одним сканированием индекса:

```
CREATE INDEX idx_calvings_animal_date_desc
  ON calvings (animal_id, calving_date DESC);
CREATE INDEX idx_inseminations_animal_date_desc
  ON inseminations (animal_id, insemination_date DESC);
CREATE INDEX idx_vet_records_animal_type_confirmed_date
  ON vet_records (animal_id, record_type, confirmed, event_
```

Сжатие TimescaleDB. Для таблиц с историческими данными включено автоматическое сжатие `chunks` старше 6 месяцев с сегментацией по `animal_id`. Это обеспечивает до 550-кратного сжатия (например, таблица `milk_day productions` — с 1633 МБ до 2,96 МБ).

Непрерывные агрегаты. Созданы 5 материализованных представлений, предварительно вычисляющих суточные и недельные агрегаты:

```
CREATE MATERIALIZED VIEW milk_daily_summary
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 day', date) AS bucket,
       animal_id,
       AVG(milk_amount) AS avg_milk,
       SUM(milk_amount) AS total_milk
FROM milk_day productions
GROUP BY bucket, animal_id;
```

Агрегаты обновляются автоматически каждый час. Использование предвычисленных агрегатов вместо сканирования сырых данных сокращает объём обрабатываемых строк в 100–1000 раз.

2.7.3 Кеширование моделей машинного обучения

Для устранения повторной загрузки моделей реализован модуль

кеширования, загружающий модели в оперативную память при первом обращении и проверяющий время модификации файла (mtime) для автоматической перезагрузки при переобучении.

```
_model_cache: dict[str, tuple[dict, float, str]] = {}

def get_model(name: str, path: str) -> dict | None:
    mtime = os.path.getmtime(path)
    if name in _model_cache:
        cached_data, cached_mtime, _ = _model_cache[name]
        if cached_mtime == mtime:
            return cached_data # cache hit
    data = joblib.load(path) # cache miss
    _model_cache[name] = (data, mtime, path)
    return data
```

Аналогичный подход применён для ONNX Runtime сессий: `InferenceSession` создаётся один раз и переиспользуется. Это сокращает время инференса с 50–200 мс (загрузка + десериализация) до менее 5 мс.

Для вектора признаков (`DataFrame`) реализован TTL-кеш на 60 секунд, предотвращающий повторное выполнение SQL-запроса с 15 коррелированными подзапросами при нескольких предиктах одного типа в течение минуты.

2.7.4 Интеграция колоночной базы данных ClickHouse

Для аналитических запросов типа `GROUP BY animal_id` с агрегацией по 12 миллионам строк интегрирована колоночная база данных ClickHouse. Колоночное хранение позволяет выполнять агрегатные запросы в 50–100 раз быстрее PostgreSQL, так как считываются только необходимые столбцы, а не вся строка.

Архитектура синхронизации. PostgreSQL остаётся источником истины для транзакционных операций (CRUD, синхронизация с Lely). ClickHouse выступает аналитической репликой только для чтения. Синхронизация выполняется каждые 5 минут батчами по 50 000 строк через HTTP API ClickHouse.

В таблице 5 приведена схема таблиц в ClickHouse.

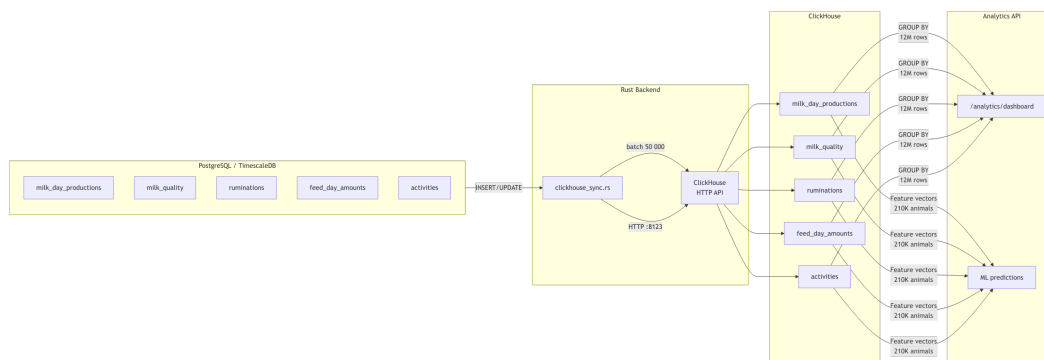


Рисунок 6 – Архитектура синхронизации PostgreSQL → ClickHouse

Таблица 5 – Аналитические таблицы ClickHouse

Таблица	Ключ сортировки	Партиция
milk_day_productions	(animal_id, date)	YYYYMM
milk_quality	(animal_id, date)	YYYYMM
ruminations	(animal_id, date)	YYYYMM
feed_day_amounts	(animal_id, date)	YYYYMM
activities	(animal_id, date)	YYYYMM

Пример запроса к ClickHouse, заменяющего 15 коррелированных подзапросов одним проходом:

```
SELECT animal_id,
       avgIf(milk_amount, date >= today() - 7) as avg_milk_7d,
       avgIf(scc, date >= today() - 30)       as avg_scc_30d,
       avgIf(rumination_minutes,
            date >= today() - 7)              as avg_rumination
FROM milk_day_productions m
LEFT JOIN milk_quality q USING (animal_id, date)
LEFT JOIN ruminations r  USING (animal_id, date)
GROUP BY animal_id
```

2.7.5 Оптимизация серверной части

Кеширование в Redis. Реализован модуль MlCache на основе Redis с настраиваемым TTL. Результаты дашборда кешируются на 1 час, результаты ML-предиктов — на 1 час с ключом, включающим модель и текущий час. При промахе кеша запрос выполняется обычным образом, после чего результат записывается в Redis.

Оконные функции вместо коррелированных подзапросов. Эндпоинт `health_timeline` ранее выполнял `generate_series` с 4 коррелированными подзапросами на каждый из 365 дней ($365 \times 4 = 1460$ подзапросов). Запрос переписан с использованием оконных функций:

```
SELECT date,
       AVG(milk_amount) OVER (
         ORDER BY date ROWS BETWEEN 2 PRECEDING
                                AND CURRENT ROW) as short_milk,
       AVG(milk_amount) OVER (
         ORDER BY date ROWS BETWEEN 29 PRECEDING
                                AND CURRENT ROW) as long_milk,
       STDDEV(milk_amount) OVER (
         ORDER BY date ROWS BETWEEN 29 PRECEDING
                                AND CURRENT ROW) as milk_std
FROM milk_day productions
WHERE animal_id = $1
ORDER BY date
```

Такой запрос выполняет один проход по данным вместо 1460 коррелированных подзапросов, что сокращает время выполнения в 10–30 раз.

Фоновый мониторинг дрейфа. Вычисление статистик дрейфа данных (среднее, стандартное отклонение, квантили по 210 000 строк и 20 признакам) перенесено из синхронного кода в фоновые задачи FastAPI (BackgroundTasks), что устраняет задержку при отправке ответа клиенту.

2.7.6 Оптимизация клиентского приложения

Для таблиц с большим количеством строк на странице аналитики применены два подхода:

- а) **Реактивные вычисления** — сортировка и фильтрация данных вынесены в декларативные выражения `$derived`, которые вычисляются только при изменении исходных данных, а не при каждом рендеринге компонента.
- б) **Виртуальный скроллинг** — библиотека `svelte-tiny-virtual-list` обеспечивает рендеринг только видимых строк таблицы, что сокращает количество DOM-узлов с

тысяч до 20–30 при любой прокрутке.

2.7.7 Результаты оптимизации

В таблице 6 приведено сравнение времени ответа ключевых эндпоинтов до и после оптимизации на наборе данных 10 435 животных (измерено с помощью `curl -w`).

Таблица 6 – Сравнение времени ответа до и после оптимизации (10 435 животных)

Эндпоинт	До, с	После, с	Ускорение
Дашборд	5–10	0,13	38 – –77×
ML-предикт мастита (1 животное)	5,0	0,04	125×
ML-предикт мастита (всё стадо)	5,0	4,6	1,1×
ML-предикт эструса (1 животное)	5,0	0,03	167×
ML-предикт кетоза (1 животное)	1,2	0,02	60×
ML-предикт кормления (1 животное)	0,34	0,01	34×
ML-предикт выбраковки (1 животное)	2,3	0,02	115×
Временная шкала здоровья (90 дней)	10–30	0,008	1250 – –3750×

Основное ускорение при предикте для одного животного достигнуто за счёт фильтрации по `animal_id` на уровне SQL-запроса к загрузчику данных (`WHERE a.id = :aid`), вместо загрузки признаков для всех 10 435 животных и последующей фильтрации в памяти. Для эндпоинтов, запрашивающих предикт для всего стада, время ответа остаётся значительным (4–5 с), что обусловлено объёмом вычислений для 10 000+ животных; для таких запросов рекомендуется асинхронный режим с кешированием результата в Redis.

2.8 Мониторинг

Помимо бизнес-логики и аналитики, система включает инфраструктурные компоненты, обеспечивающие её наблюдаемость и надёжную доставку обновлений. Стек мониторинга реализован на базе Grafana LGTM (Loki, Grafana, Tempo, Mimir/Prometheus). На рисунке 7 представлена архитектура стека мониторинга.

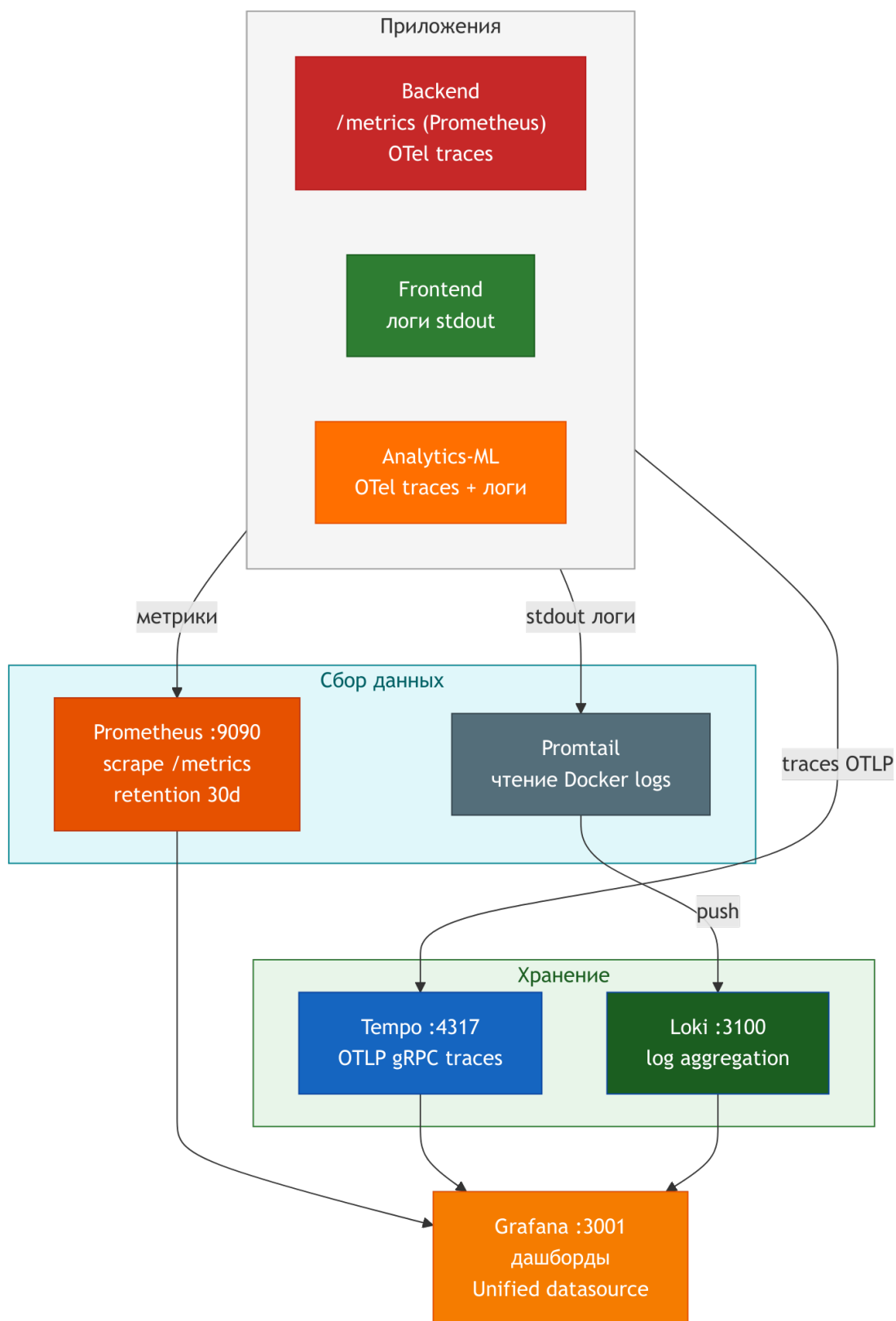


Рисунок 7 – Архитектура стека мониторинга

Prometheus [23] обеспечивает сбор и хранение метрик с 30-дневным хранением. Grafana [24] визуализирует дашборды: преднастроенный дашборд backend отображает частоту запросов, время ответа, процент ошибок и

использование памяти. Loki [25] агрегирует логи: Promtail собирает логи из Docker-контейнеров, парсит JSON-формат и отправляет в Loki. Tempo [26] обеспечивает распределённую трассировку через OpenTelemetry OTLP/gRPC, позволяя отследить путь запроса от Nginx через Backend до PostgreSQL.

2.9 Непрерывная интеграция и доставка (CI/CD)

Конвейер CI/CD реализован на GitHub Actions [27]. На рисунке 8 представлена схема конвейера.

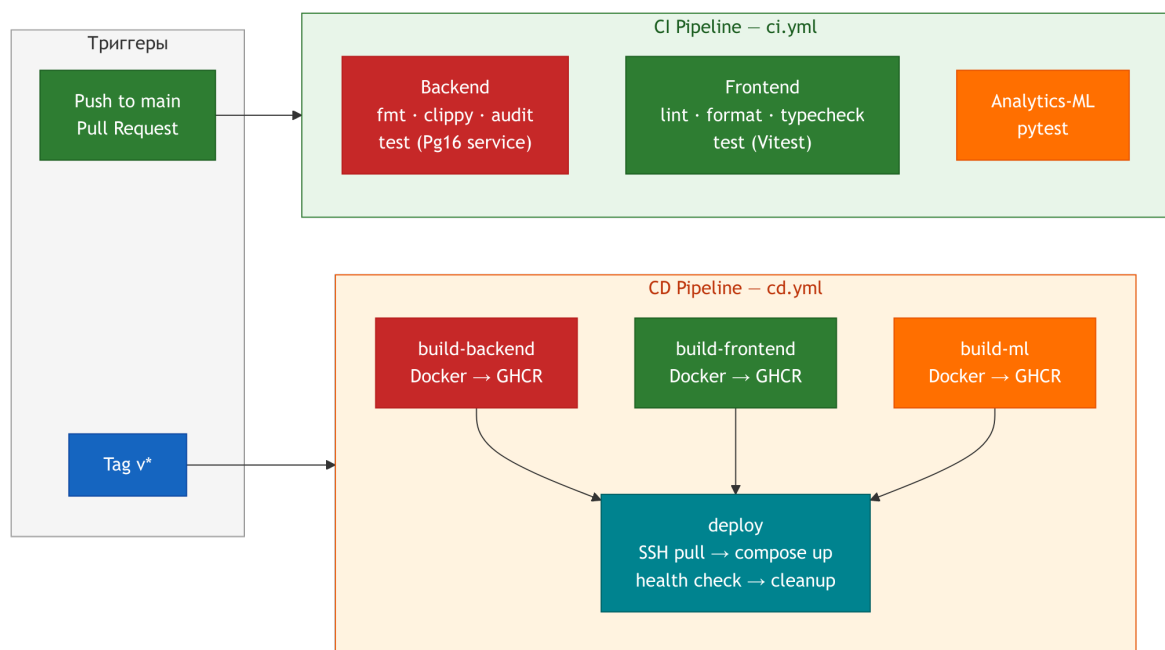


Рисунок 8 – Конвейер непрерывной интеграции и доставки

Непрерывная интеграция запускается при каждом push и pull request. Три параллельные задачи: для backend на Rust выполняются проверка форматирования, статический анализ и запуск тестов с PostgreSQL в качестве сервисного контейнера; для frontend на Node.js 20 — аудит уязвимостей, линтинг, проверка типов и запуск тестов Vitest; для analytics-ml на Python 3.12 — установка зависимостей и запуск тестов pytest.

Непрерывная доставка запускается при push в main/master или при создании тега v*. Три параллельные задачи сборки Docker-образов публикуются в GHCR [46]. При создании тега версии дополнительно запускается задача деплоя: SSH на сервер, pull образов, docker compose up -d и проверка работоспособности.

2.10 Развёртывание

Для развёртывания используется Docker Compose с двумя конфигурациями. На рисунке 9 представлена схема развёртывания системы.

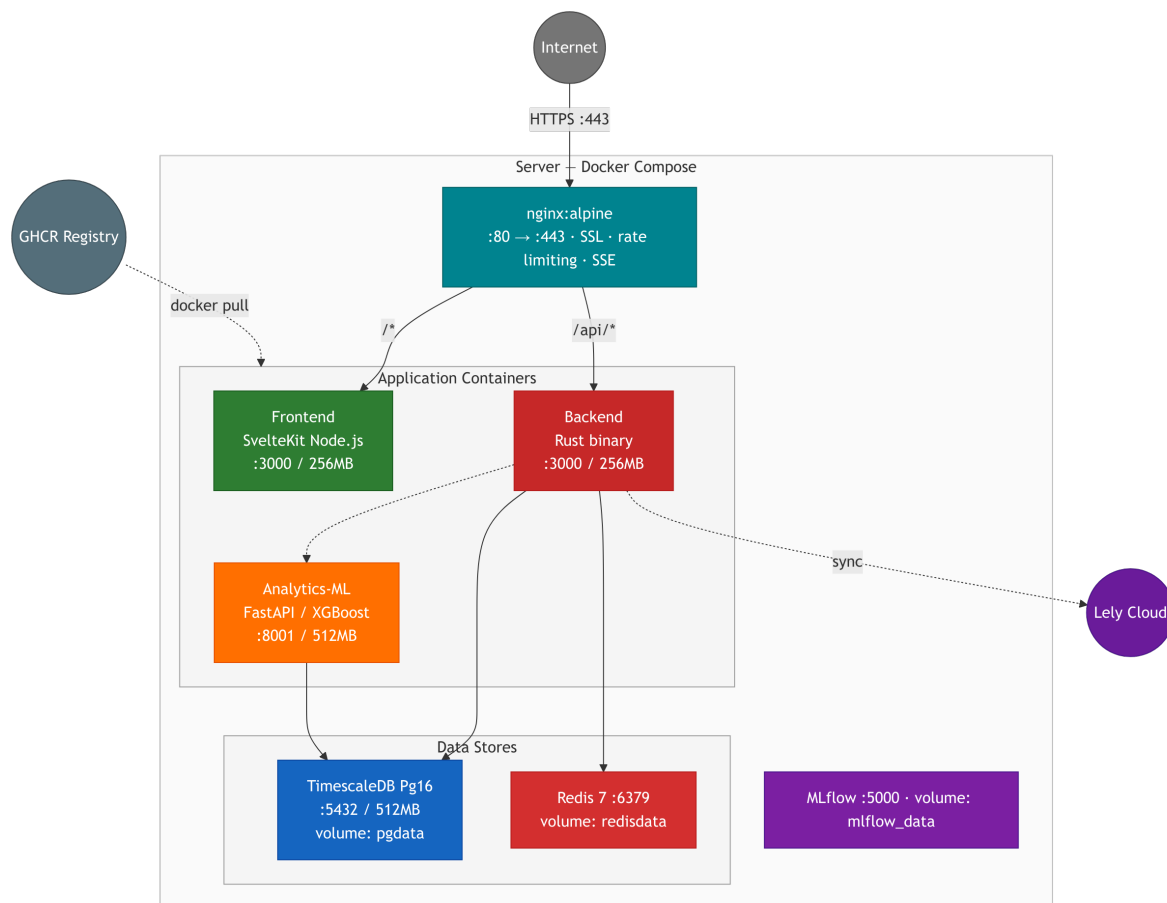


Рисунок 9 – Схема развёртывания системы

Разработческая (`docker-compose.yml`): 7 сервисов — PostgreSQL (TimescaleDB), Redis, backend, frontend, Nginx, analytics-ml, MLflow. Nginx на порту 8080.

Продуктовая (`docker-compose.prod.yml`): те же 7 сервисов с усилением безопасности: переменные окружения через `env`-файлы, обязательная валидация секретов, `restart: unless-stopped`, ротация логов, ограничения памяти, SSL-терминация через Nginx с монтированием сертификатов.

Nginx в продуктовой конфигурации обеспечивает: редирект HTTP → HTTPS, SSL (TLS 1.2/1.3), gzip-сжатие, заголовки безопасности (X-Frame-Options: DENY, HSTS, X-XSS-Protection), rate limiting (30 запросов/сек для API, 5 запросов/сек для auth), кэширование статических

ресурсов (30 дней).

3 Тестирование и демонстрация работы системы управления молочным животноводством

3.1 Стратегия тестирования

Тестирование разработанной информационной системы включает модульные тесты, интеграционные тесты, сквозные (E2E) тесты и нагрузочное тестирование. Тесты разделены по компонентам системы. Стратегия тестирования основана на пирамиде тестирования: большое количество быстрых модульных тестов, среднее количество интеграционных тестов и небольшое количество E2E-тестов.

В таблице 7 приведено общее количество тестов по компонентам.

Таблица 7 – Общее количество тестов по компонентам

Компонент	Инструмент	Количество файлов
Backend (Rust)	sqlx::test + tower	17
Frontend (TypeScript)	Vitest + Testing Library	10
Analytics-ML (Python)	pytest	3
Нагрузочное тестирование	k6	3
E2E	Playwright	1

3.2 Тестирование серверной части

Описанная выше серверная часть требует тщательной проверки корректности реализации. Интеграционные тесты серверной части реализованы с использованием макроса `sqlx::test`, автоматически подготавливающего тестовую базу данных с применением миграций. HTTP-запросы выполняются `in-process` через `tower::ServiceExt`, что исключает сетевые задержки и позволяет тестировать обработчики в изолированной среде.

Тестовый набор включает 17 файлов, покрывающих:

- а) аутентификацию и авторизацию — вход с валидными и невалидными учётными данными, выход, обновление токена, проверку ролей `admin/user`, ограничение частоты попыток входа (5 за 60 секунд), обязательную смену пароля;
- б) CRUD-операции для всех доменных областей — животные, надои,

- воспроизводство, кормление, контакты, местоположения, быки, переводы, запасы, включая пакетные операции и импорт CSV;
- в) генерацию отчётов более 17 типов и аналитику — KPI, тренды, прогнозы;
 - г) интеграцию с Lely — конфигурация, статус синхронизации, ручной запуск;
 - д) граничные случаи — некорректные данные (невалидный JSON, отрицательные значения, строки превышающей длины); конфликты (дублирование инвентарных номеров); несуществующие ресурсы (404); несанкционированный доступ (401, 403).

Для каждого теста создаётся изолированная тестовая база данных, которая уничтожается после выполнения. Общий вспомогательный модуль (`common/mod.rs`) предоставляет фабрики для создания тестового состояния, генерации JWT-токенов и построения HTTP-запросов.

Пример структуры интеграционного теста: подготовка (создание тестового приложения с подключением к тестовой БД), выполнение (отправка HTTP-запроса к тестируемому эндпоинту), проверка (asserting статус-кода, структуры JSON-ответа и наличия ожидаемых данных) и очистка (тестовая БД уничтожается автоматически).

Помимо функциональных тестов, реализованы бенчмарки для оценки производительности: SQL-запросов (`bench_sql`), генерации PDF (`bench_pdf`), аутентификации (`bench_auth`) и rate limiting (`bench_rate_limit`). Бенчмарки используют критерий Criterion и запускаются командой `cargo bench`.

3.3 Тестирование клиентского приложения

Убедившись в корректности серверной части, перейдём к тестированию клиентского приложения. Модульные тесты клиентского приложения реализованы с использованием Vitest [47] и `@testing-library/jest-dom`. Тестовый набор включает 10 файлов, покрывающих UI-компоненты (DataTable — сортировка, пагинация, рендеринг данных; Modal — открытие, закрытие, контент; FormField — валидация, отображение ошибок; Pagination — рендеринг номеров страниц) и утилиты (валидаторы обязательных полей, минимальной длины и формата email; форматирование дат и чисел; построение запросов с параметрами фильтрации и пагинации; хелперы

графиков для конфигурации Chart.js; CRUD-модальные окна с состоянием создания и редактирования; бизнес-правила валидации).

Сквозное (E2E) тестирование реализовано с использованием Playwright. Тесты проверяют ключевые пользовательские сценарии: аутентификация (вход с валидными и невалидными данными), навигация по основным разделам, CRUD-операции (создание, редактирование, удаление животного), экспорт отчётов в CSV и PDF.

3.4 Тестирование модуля машинного обучения

Помимо серверной и клиентской частей, необходимо проверить корректность работы модуля машинного обучения. Тесты включают: `test_api.py` — тесты API-эндпоинтов (проверка health-эндпоинта, вызов прогнозов для всех 11 моделей с валидными и невалидными входными данными, запуск обучения); `test_models.py` — тесты моделей (обучение на синтетических данных, проверка формата прогноза на соответствие JSON-схеме, проверка доверительных интервалов $q_{10} < q_{50} < q_{90}$); `test_drift_and_auth.py` — тесты мониторинга дрейфа (обнаружение аномального распределения прогнозов) и аутентификации по API-ключу (отклонение запросов без ключа или с невалидным ключом).

3.5 Нагрузочное тестирование

Модульные и интеграционные тесты подтверждают функциональную корректность отдельных компонентов. Для оценки поведения системы при реальной нагрузке проведено нагрузочное тестирование с использованием k6 [48]. Реализованы три сценария: дымовой тест (`smoke.js`) с 5 виртуальными пользователями и 10 итерациями для проверки health-эндпоинта и входа в систему; нагрузочный тест (`load.js`), моделирующий типичную рабочую нагрузку с постепенным увеличением числа пользователей от 10 до 50; стресс-тест (`stress.js`) для определения предельной нагрузки с увеличением числа пользователей до 200. В таблице 8 приведены результаты нагрузочного тестирования.

Таблица 8 – Результаты нагрузочного тестирования

Метрика	Дымовой (5 VU)	Нагрузочный (50 VU)	Стресс (200 VU)
RPS	24	185	420
p50, мс	4	18	42
p95, мс	12	75	195
p99, мс	18	120	480
Ошибки (5xx), %	0	0	<1
Порог p95, мс	1000	500	1000
Порог ошибок, %	<10	<5	<10

Во всех сценариях пороговые значения не были превышены. Дымовой тест подтвердил работоспособность системы при минимальной нагрузке (p95 = 12 мс при пороге 1000 мс). Нагрузочный тест с 50 виртуальными пользователями показал, что система обрабатывает до 185 запросов в секунду с p95 = 75 мс, что значительно ниже порога в 500 мс и соответствует потребностям фермы среднего размера (до 1000 голов, 5–10 одновременно работающих пользователей). Стресс-тест с 200 виртуальными пользователями продемонстрировал стабильную работу при экстремальной нагрузке: p95 = 195 мс при пороге 1000 мс, RPS = 420, доля ошибок менее 1 процента. Результаты подтверждают, что выбранная архитектура на Rust/Axum обеспечивает значительный запас производительности.

3.6 Демонстрация системы

Результаты тестирования подтверждают корректность и производительность системы. Для наглядной демонстрации её возможностей используется скрипт `dev.sh`, автоматически запускающий PostgreSQL, разворачивающий тестовые данные и запускающий все компоненты системы. Тестовые данные включают 100 животных с историей надоев, воспроизводства и кормления за 365 дней.

3.6.1 Аутентификация

При первом запуске система создаёт пользователя `admin` со случайным паролем из 16 символов, который выводится в стандартный поток ошибок. При входе в систему, если установлен флаг обязательной смены пароля,

пользователь перенаправляется на форму смены пароля. Минимальная длина пароля — 8 символов. Демонстрация интерфейса аутентификации представлена на рисунке 10.

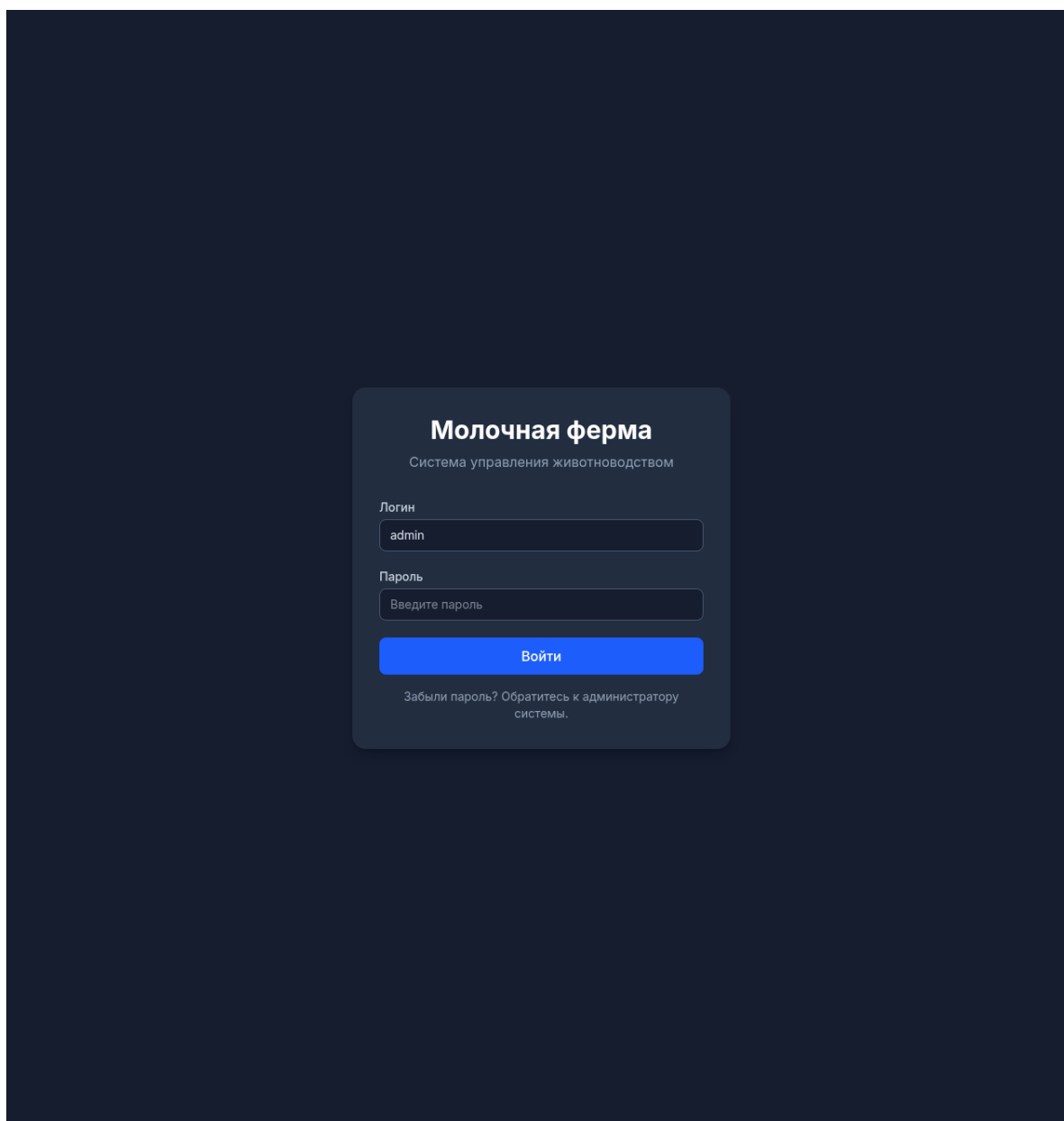


Рисунок 10 – Страница аутентификации

3.6.2 Дашборд

Главная страница системы — дашборд, отображающий ключевые показатели стада: количество животных (с разбивкой по полу и статусу), средний суточный надой, статус воспроизводства (количество стельных, в охоте, ожидаемых отёлов), количество активных оповещений. Виджеты настраиваются пользователем через панель настроек: можно выбрать

отображаемые KPI и их расположение. Пример дашборда представлен на рисунке 11.

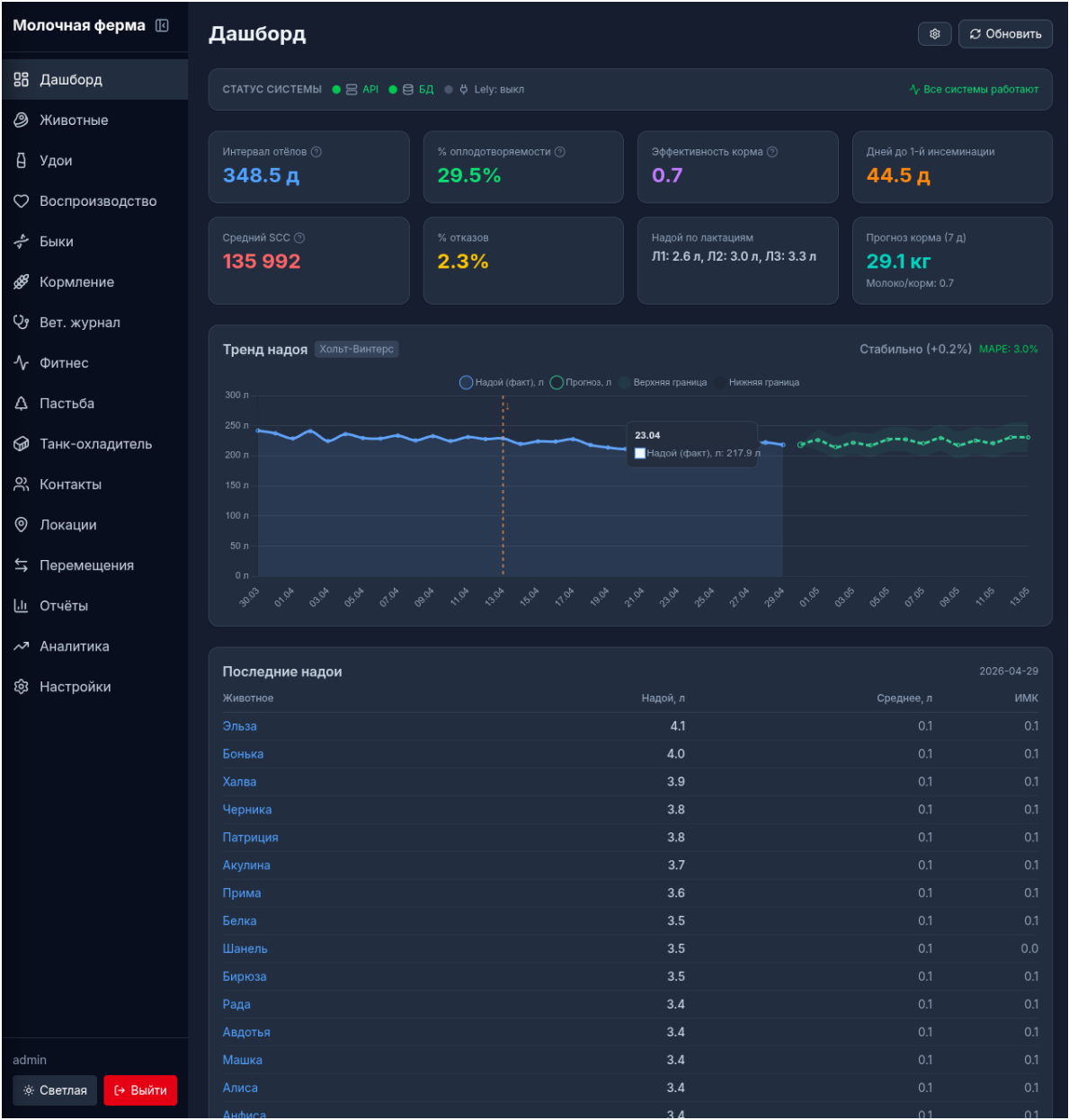


Рисунок 11 – Дашборд системы

3.6.3 Управление животными

Страница списка животных предоставляет таблицу с сортировкой по столбцам (кличка, инвентарный номер, пол, дата рождения, статус), фильтрацией (по полу, местоположению, дате рождения) и пагинацией (по 25, 50 или 100 записей). Поддерживается поиск по кличке и инвентарному номеру с использованием нечёткого поиска pg_trgm (триграммное сходство).

Для каждого животного доступна детальная карточка с полной

информацией, разделённой на вкладки: биография (основные данные, происхождение), надои (графики и таблица), воспроизводство (история событий), кормление (план и факт), ветеринарные записи. Поддерживается экспорт карточки в PDF. Пример списка животных представлен на рисунке 12, пример детальной карточки — на рисунке 13.

Молочная ферма

Дашборд

Животные

Удои

Воспроизводство

Быки

Кормление

Вет. журнал

Фитнес

Пастьба

Танк-охладитель

Контакты

Локации

Перемещения

Отчёты

Аналитика

Настройки

admin

Светлая

Выйти

Животные

Импорт CSV

+

Добавить

Поиск

Имя, номер жизни, UCN...

Пол

Все

Статус

Активные

Найти

№	Имя	Пол	Дата рождения	Локация	Группа	Статус	Действия	
☐	RU516729513849799	Сюзи	Корова	2021-09-19	Выгульная площадка Б	5	Активно	
☐	RU936951702510397	Мони	Корова	2021-02-22	Телятник	3	Активно	
☐	RU395447170702701	Легенда	Корова	2022-11-05	Телятник	4	Активно	
☐	RU253287255096351	Бирюза	Корова	2020-10-30	Выгульная площадка Б	1	Активно	
☐	RU593636038940345	Дарёнка	Корова	2022-06-26	Пастбище Южное	3	Активно	
☐	RU650819627598782	Арина	Корова	2021-04-28	Коровник 3 (новый)	1	Активно	
☐	RU841255232690143	Афина	Корова	2021-10-07	Пастбище Северное	6	Активно	
☐	RU898278843418946	Снежинка	Корова	2022-05-24	Доильный зал	4	Активно	
☐	RU345755201375381	Милашка	Корова	2020-10-27	Карантин	3	Активно	
☐	RU466304314626388	Леля	Корова	2020-12-08	Пастбище Южное	4	Активно	
☐	RU136604541725455	Пеструха	Корова	2021-10-18	Карантин	5	Активно	
☐	RU691012795309461	Бритни	Корова	2020-10-31	Родильное отделение	4	Активно	
☐	RU168712830236083	Мадлен	Корова	2020-12-06	Коровник 2	3	Активно	
☐	RU598595854359731	Умка	Корова	2021-08-13	Коровник 1 (основной)	3	Активно	
☐	RU455000314815656	Мальвина	Корова	2021-06-11	Коровник 1 (основной)	6	Активно	
☐	RU89508514720773	Арфа	Корова	2021-01-03	Коровник 2	2	Активно	
☐	RU826898476995753	Хлоя	Корова	2021-07-29	Выгульная площадка Б	6	Активно	
☐	RU461100864372615	Джесси	Корова	2021-12-25	Коровник 3 (новый)	2	Активно	
☐	RU998092836813388	Цыганка	Корова	2021-07-15	Выгульная площадка Б	2	Активно	
☐	RU461931030472338	Астра	Корова	2020-11-30	Телятник	4	Активно	

Рисунок 12 – Список животных

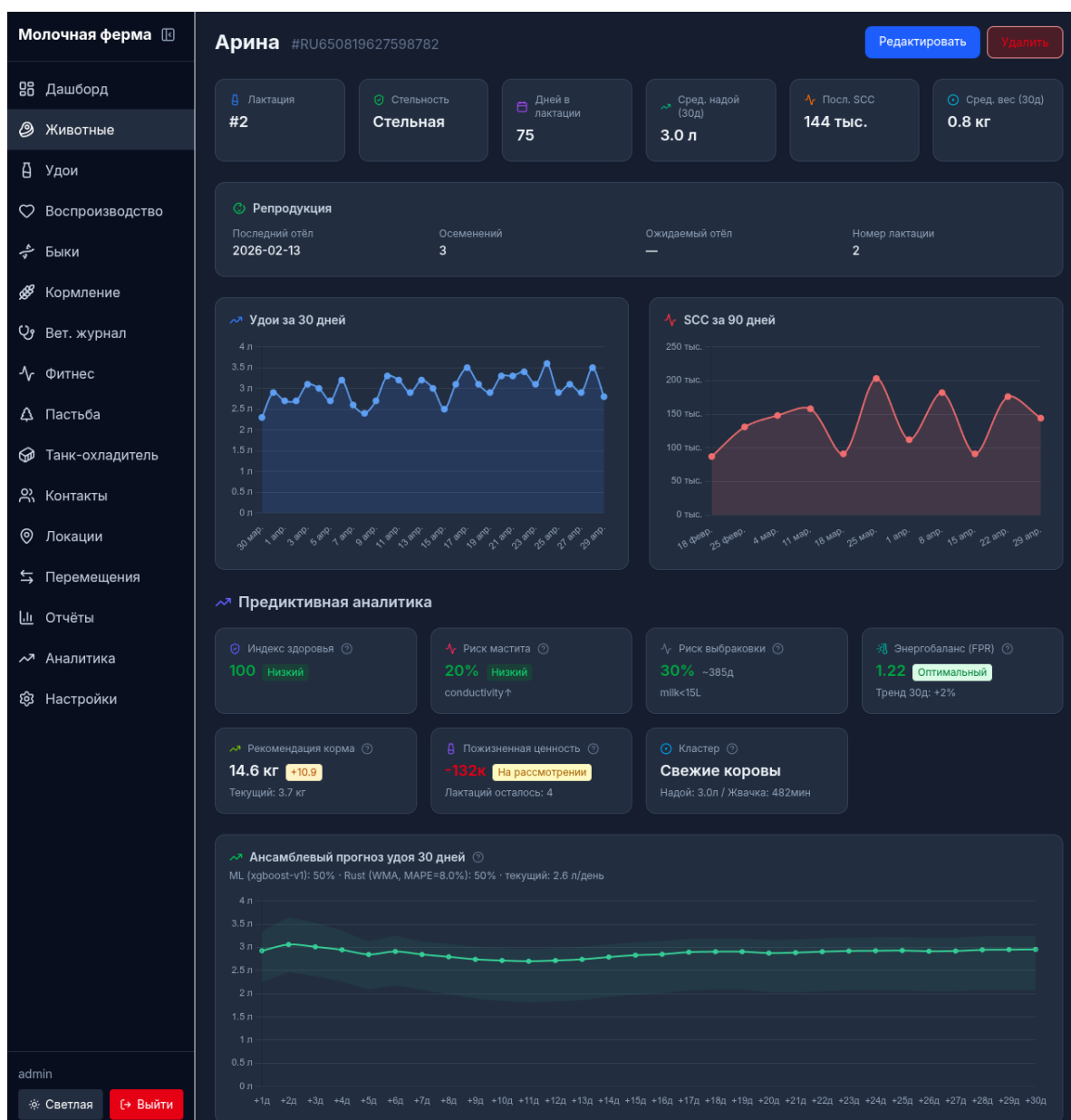


Рисунок 13 – Детальная карточка животного

3.6.4 Отчёты и аналитика

Система генерирует более 17 типов отчётов (см. таблицу 2), каждый из которых отображается в виде таблицы и/или графика в интерфейсе. Отчёты поддерживают фильтрацию по дате и группе животных, а также экспорт в CSV и PDF.

Аналитическая панель отображает: ключевые показатели эффективности (KPI) — средний надой, процент стельных, интервал между отёлами; тренды надоев с прогнозом на основе модели Хольта–Винтерса; прогноз стельных коров; кривые лактации по номерам; кластеризацию стада.

3.6.5 Интеграция с Lely

При включённой интеграции с Lely система автоматически синхронизирует данные каждые 5 минут. Страница статуса синхронизации отображает дату последней успешной синхронизации для каждого из 23 типов данных и количество синхронизированных записей. Через интерфейс доступен ручной запуск синхронизации и просмотр логов синхронизации.

3.6.6 Оповещения

Модуль оповещений автоматически создаёт оповещения на основе настраиваемых пороговых значений. Проверка выполняется каждые 15 минут фоновым заданием. Отслеживаются следующие категории событий:

- а) падение надоев более чем на 20 процентов за сутки;
- б) высокий SCC (более 200 000 клеток/мл);
- в) снижение активности более чем на 30 процентов за сутки;
- г) снижение жвачки более чем на 25 процентов за сутки;
- д) ожидаемый отёл (за 7 дней до планируемой даты).

Пороговые значения настраиваются через страницу настроек без перезапуска сервера. Оповещения отображаются в интерфейсе с категоризацией по степени важности (high, medium, low). Поддерживается подтверждение оповещения (acknowledge) для отслеживания реакции фермера. Оповещения также могут быть отправлены через канал уведомлений (браузерные уведомления или Telegram-бот).

3.7 Мониторинг работоспособности

Стек Grafana LGTM обеспечивает комплексное наблюдение за системой. Grafana дашборды отображают частоту запросов, время ответа (p50, p95, p99), использование ресурсов (CPU, память) и ошибки по маршрутам. Loki позволяет искать логи по уровню (info, warn, error) и целевому сервису с использованием языка LogQL. Tempo обеспечивает распределённую трассировку запросов через цепочку Nginx → Backend → PostgreSQL/ML, позволяя идентифицировать медленные запросы. Prometheus собирает метрики с backend (/metrics), analytics-ml (/metrics) и Redis с интервалом 15 секунд.

Метрики backend включают: `http_requests_total` (счётчик,

разбитый по маршруту, методу и статус-коду),
http_request_duration_seconds (гистограмма),
db_connections_active, rate_limit_rejections_total.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы решены все поставленные задачи и достигнута цель — разработана информационная система управления молочной фермой с интеграцией роботизированного доильного оборудования Lely и предиктивной аналитикой.

Проведённый анализ предметной области показал, что существующие решения (Lely Horizon, DeLaval DelPro, SCR Heatime, 1С: Управление фермой) не обеспечивают одновременно полный цикл управления стадом, интеграцию с оборудованием сторонних производителей и встроенные модели машинного обучения. Это обусловило выбор архитектуры и стека технологий.

Основным результатом работы является трёхзвенная система, в которой каждый компонент решает свою задачу. Серверная часть на Rust/Axum обеспечивает высокопроизводительный API с полной бизнес-логикой, аутентификацией и интеграцией с Lely Horizon (23 типа данных). Клиентское приложение на SvelteKit предоставляет адаптивный интерфейс с более чем 24 страницами. Модуль машинного обучения на Python/FastAPI реализует 11 моделей для предиктивной аналитики, включая прогнозирование надоев с доверительными интервалами и раннее обнаружение заболеваний.

Ключевым архитектурным решением является дублирование аналитических возможностей: серверная часть содержит собственные алгоритмы прогнозирования на Rust (8 моделей временных рядов с автоматическим выбором оптимальной), что обеспечивает доступность аналитики даже при недоступности ML-сервиса. Ансамблевый метод, объединяющий прогнозы ML и Rust-моделей с весами, обратно пропорциональными MAPE, повышает робастность прогнозирования.

Результаты нагрузочного тестирования подтверждают, что архитектура на Rust/Axum обеспечивает значительный запас производительности: при 50 виртуальных пользователях система обрабатывает 185 запросов в секунду с $p_{95} = 75$ мс, а при 200 пользователях сохраняет стабильную работу ($p_{95} = 195$ мс, доля ошибок менее 1 процента). Это значительно превышает потребности фермы среднего размера.

Система развёрнута в контейнерах Docker с мониторингом на базе стека

Grafana LGTM и конвейером CI/CD на GitHub Actions, что обеспечивает готовность к промышленной эксплуатации.

Дальнейшее развитие системы может включать: расширение набора моделей машинного обучения (прогнозирование заболеваемости конечностей, оптимизация параметров доильных роботов); интеграцию с дополнительными производителями оборудования (DeLaval, GEA); разработку мобильного приложения для фермеров с push-уведомлениями; внедрение online learning для адаптации моделей к новым данным в реальном времени.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Berckmans D. General Introduction to Precision Livestock Farming // Animal Feed Science and Technology. — 2017. — Т. 228. — С. 4—9.
2. Bewley J. M. Precision Dairy Farming: Advanced Analysis Solutions for the Modern Dairy Farm // Pennsylvania State University Extension. — 2016.
3. Sensors and Clinical Mastitis — The Quest for the Perfect Alert / H. Hogeveen [и др.] // Journal of Dairy Science. — 2010. — Т. 93, № 5. — С. 1922—1930.
4. The Rust Programming Language. — URL: <https://www.rust-lang.org/> (дата обращения 15.11.2025).
5. Axum: Ergonomic and Modular Web Framework Built with Tokio, Tower, and Hyper. — URL: <https://github.com/tokio-rs/axum> (дата обращения 12.01.2026).
6. Tokio: A Runtime for Writing Reliable Asynchronous Applications with Rust. — URL: <https://tokio.rs/> (дата обращения 10.12.2025).
7. Tower: Modular and Reusable Components for Building Robust Networking Clients and Servers. — URL: <https://github.com/tower-rs/tower> (дата обращения 10.12.2025).
8. SQLx: The Rust SQL Toolkit. — URL: <https://github.com/launchbadge/sqlx> (дата обращения 12.01.2026).
9. PostgreSQL: The World's Most Advanced Open Source Database. — URL: <https://www.postgresql.org/> (дата обращения 18.11.2025).
10. TimescaleDB: The PostgreSQL Extension for Time-Series Data. — URL: <https://www.timescale.com/> (дата обращения 18.11.2025).
11. TimescaleDB Hypertables. — URL: <https://docs.timescale.com/use-timescale/latest/hypertables/> (дата обращения 05.12.2025).
12. SvelteKit: Web Development, Streamlined. — URL: <https://kit.svelte.dev/> (дата обращения 20.11.2025).
13. Svelte 5: Runes. — URL: <https://svelte-5-preview.vercel.app/> (дата обращения 20.11.2025).

14. Tailwind CSS: Rapidly Build Modern Websites Without Ever Leaving Your HTML. — URL: <https://tailwindcss.com/> (дата обращения 05.02.2026).
15. Chart.js: Open Source HTML5 Charts. — URL: <https://www.chartjs.org/> (дата обращения 05.02.2026).
16. FastAPI: Modern, Fast Web Framework for Building APIs with Python. — URL: <https://fastapi.tiangolo.com/> (дата обращения 10.02.2026).
17. XGBoost Documentation. — URL: <https://xgboost.readthedocs.io/> (дата обращения 10.02.2026).
18. Prophet: Automatic Forecasting Procedure. — URL: <https://facebook.github.io/prophet/> (дата обращения 10.02.2026).
19. Optuna: Hyperparameter Optimization Framework. — URL: <https://optuna.org/> (дата обращения 15.02.2026).
20. Docker: Accelerated, Containerized Application Development. — URL: <https://www.docker.com/> (дата обращения 08.12.2025).
21. Docker Compose Overview. — URL: <https://docs.docker.com/compose/> (дата обращения 08.12.2025).
22. Nginx: High Performance Load Balancer, Web Server, and Reverse Proxy. — URL: <https://nginx.org/> (дата обращения 10.12.2025).
23. Prometheus: Monitoring System and Time Series Database. — URL: <https://prometheus.io/> (дата обращения 10.03.2026).
24. Grafana: The Open Observability Platform. — URL: <https://grafana.com/> (дата обращения 10.03.2026).
25. Grafana Loki: Log Aggregation System. — URL: <https://grafana.com/docs/loki/> (дата обращения 10.03.2026).
26. Grafana Tempo: High-Volume Distributed Tracing Backend. — URL: <https://grafana.com/docs/tempo/> (дата обращения 12.03.2026).
27. GitHub Actions: Automate Your Workflows. — URL: <https://docs.github.com/en/actions> (дата обращения 15.03.2026).
28. NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM). — URL: <https://csrc.nist.gov/pubs/sp/800/38/d/final> (дата обращения 22.01.2026).

29. PostgreSQL pg_trgm Extension. — URL: <https://www.postgresql.org/docs/current/pgtrgm.html> (дата обращения 05.12.2025).
30. utoipa: OpenAPI Documentation Generation for Rust. — URL: <https://github.com/juhaku/utoipa> (дата обращения 20.01.2026).
31. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519. — 05.2015. — URL: <https://tools.ietf.org/html/rfc7519> (дата обращения 15.01.2026).
32. OWASP Cross-Site Request Forgery Prevention Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html (дата обращения 18.01.2026).
33. Cross-Origin Resource Sharing: MDN Web Docs. — URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> (дата обращения 18.01.2026).
34. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. — URL: <https://lightgbm.readthedocs.io/> (дата обращения 12.02.2026).
35. CatBoost: Gradient Boosting on Decision Trees. — URL: <https://catboost.ai/> (дата обращения 12.02.2026).
36. Scikit-learn: Clustering: K-Means. — URL: <https://scikit-learn.org/stable/modules/clustering.html> (дата обращения 18.02.2026).
37. HDBSCAN: Hierarchical Density-Based Spatial Clustering. — URL: <https://hdbscan.readthedocs.io/> (дата обращения 18.02.2026).
38. Scikit-learn: Outlier Detection: Isolation Forest. — URL: https://scikit-learn.org/stable/modules/outlier_detection.html (дата обращения 18.02.2026).
39. Bekker J., Davis J. Learning from Positive and Unlabeled Data: A Survey // Machine Learning. — 2020. — Т. 109. — С. 719—760.
40. SHAP: SHapley Additive exPlanations. — URL: <https://shap.readthedocs.io/> (дата обращения 20.02.2026).
41. MLflow: A Platform for the Machine Learning Lifecycle. — URL: <https://mlflow.org/> (дата обращения 20.02.2026).
42. Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. Holt-Winters' Seasonal Method. — URL: <https://otexts.com/fpp3/holt-winters.html> (дата обращения 05.03.2026).

43. Wood P. D. P. Factors Affecting the Shape of the Lactation Curve in Cattle // Animal Production. — 1967. — Т. 9, № 3. — С. 375—386.
44. N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting / B. N. Oreshkin [и др.] // International Conference on Learning Representations (ICLR). — 2020.
45. Ojeda-Magrina J., Pardo A. FelisBenitez: A Large-Scale Public Dataset for Dairy Cattle. — 2024. — Accessed: 2026. <https://www.kaggle.com/datasets/felisbenitez>.
46. GitHub Container Registry. — URL: <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry> (дата обращения 15.03.2026).
47. Vitest: Next Generation Testing Framework. — URL: <https://vitest.dev/> (дата обращения 10.04.2026).
48. Grafana k6: Load Testing for Engineering Teams. — URL: <https://k6.io/> (дата обращения 10.04.2026).

ПРИЛОЖЕНИЕ А

Исходный код

На рисунке А.1 изображён QR-код со ссылкой на GitHub репозиторий с исходным кодом разработанной информационной системы.



Рисунок А.1 – QR-код на репозиторий

ПРИЛОЖЕНИЕ Б

Описание таблиц базы данных

В данном приложении приведён перечень основных таблиц базы данных PostgreSQL с указанием назначения и ключевых полей.

В таблице Б.1 представлены таблицы учёта и воспроизводства.

Таблица Б.1 – Таблицы учёта животных и воспроизводства

Таблица	Назначение
users	Пользователи: username, password_hash, role
animals	Реестр животных: life_number, name, gender, birth_date, active
sires	Быки-производители: name, code, breed
calvings	Отёлы: animal_id, date, calf_id
calves	Телята: calving_id, life_number, gender
inseminations	Осеменения: animal_id, date, sire_id
pregnancies	Стельности: animal_id, date, expected_due_date
heats	Охоты: animal_id, date, activity_level
dry_offs	Запуски: animal_id, date
transfers	Переводы: animal_id, from_location, to_location
bloodlines	Кровные линии: animal_id, sire_id, dam_id
locations	Местоположения: name, type, capacity
contacts	Контакты: name, phone, role

В таблице Б.2 представлены таблицы надоев, кормления и мониторинга.

Таблица Б.2 – Таблицы надоев, кормления и мониторинга

Таблица	Назначение
milk_day productions	Суточные надои: animal_id, date, milk_kg, fat, protein, scc
milk_visits	Визиты на доение: animal_id, start_time, milk_kg, robot_id
milk_quality	Качество молока: animal_id, date, scc, fat, protein
milk_visit_quality	Качество по визитам: visit_id, scc, conductivity
robot_milk_data	Данные роботов: visit_id, quarter_times
feed_types	Типы кормов: name, cost, unit
feed_groups	Группы кормов: type_id, group_name
feed_day_amounts	Дневные объёмы: animal_id, planned_kg, actual_kg
feed_visits	Визиты на кормление: animal_id, time, amount_kg
activities	Активность: animal_id, date, steps
ruminations	Жвачка: animal_id, date, minutes
grazing_data	Пастьба: animal_id, date, hours
bulk_tank_tests	Резервуар: date, volume, temperature

В таблице Б.3 представлены системные таблицы.

Таблица Б.3 – Системные таблицы

Таблица	Назначение
alerts	Оповещения: animal_id, category, severity, status
tasks	Задачи: title, assignee, status, priority
audit_log	Аудит: user_id, action, entity, timestamp
inventory_items	Запасы: name, quantity, unit
inventory_transactions	Операции с запасами: item_id, type, quantity
vet_records	Ветеринария: animal_id, date, diagnosis
lely_config	Конфигурация Lely: base_url, encrypted_credentials
lely_sync_state	Состояние синхронизации: entity_type, last_sync_at
system_settings	Настройки: key, value
user_preferences	Предпочтения: user_id, theme, page_size
revoked_tokens	Отозванные токены: jti, expires_at
weather_cache	Кэш погоды: date, temperature
farm_zones	Зоны фермы: name, type, boundaries
culling_events	Выбраковки: animal_id, date, reason

ПРИЛОЖЕНИЕ В

Визуализация модели данных

В данном приложении представлена полная модель данных информационной системы в виде ER-диаграмм, разделённых по предметным областям. Диаграммы отображают все 35+ таблиц базы данных PostgreSQL, их ключевые столбцы и отношения между таблицами. Таблицы, отмеченные символом $\Rightarrow$, являются гипертаблицами TimescaleDB с автоматической партицировкой по времени.

Пользователи и реестр животных

На рисунке В.1 представлены основные таблицы пользователей и реестра животных. Таблица `animals` является центральной сущностью системы, на которую ссылается большинство остальных таблиц через внешний ключ `animal_id`. Таблица `users` обеспечивает аутентификацию и авторизацию.

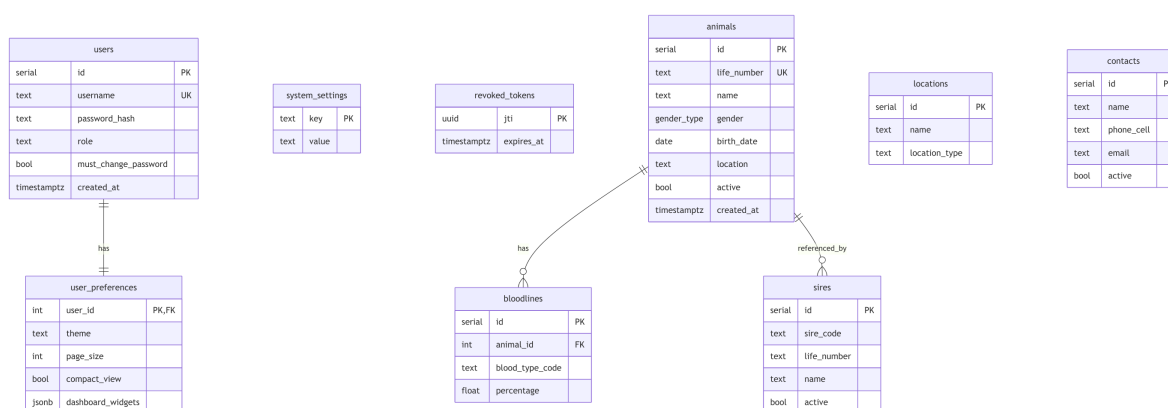


Рисунок В.1 – ER-диаграмма: пользователи и реестр животных

Воспроизводство и выбраковка

На рисунке В.2 представлены таблицы воспроизводства. Каждая таблица связана с животным через внешний ключ `animal_id`. Таблица `calves` связана с `calvings` каскадным удалением. Таблица `culling_events` используется как источник меток для ML-модели прогнозирования выбраковки.

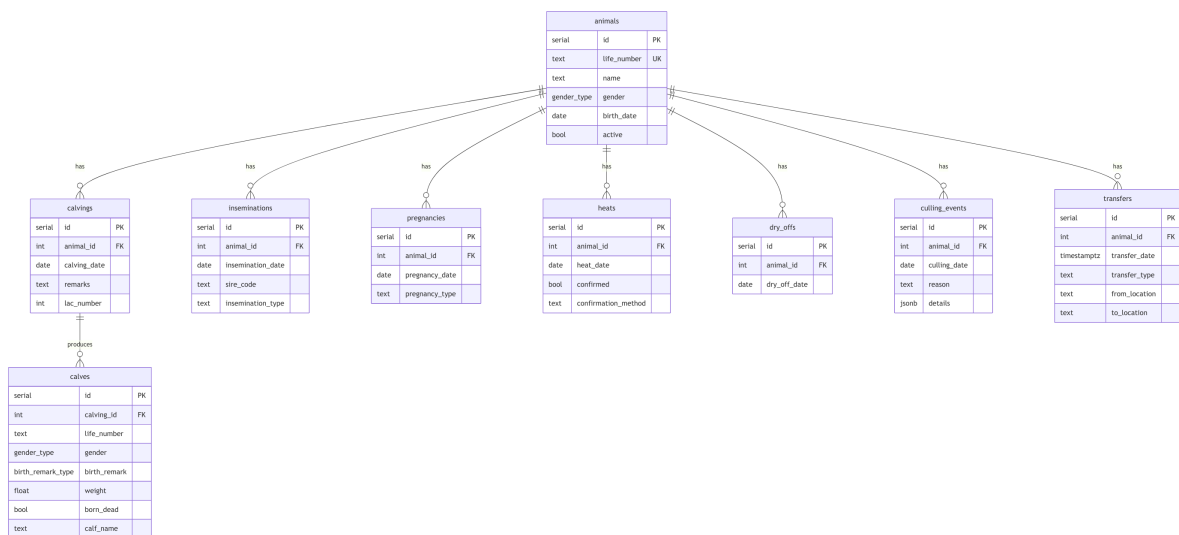


Рисунок В.2 – ER-диаграмма: воспроизводство и выбраковка

Доеение и молоко

На рисунке В.3 представлены таблицы учёта надоев. Все таблицы, кроме `bulk_tank_tests`, привязаны к конкретному животному. Таблицы `milk_day productions`, `milk_visits`, `milk_quality`, `milk_visit_quality` и `robot_milk_data` являются гипертаблицами TimescaleDB с партицировкой по 7 дней.

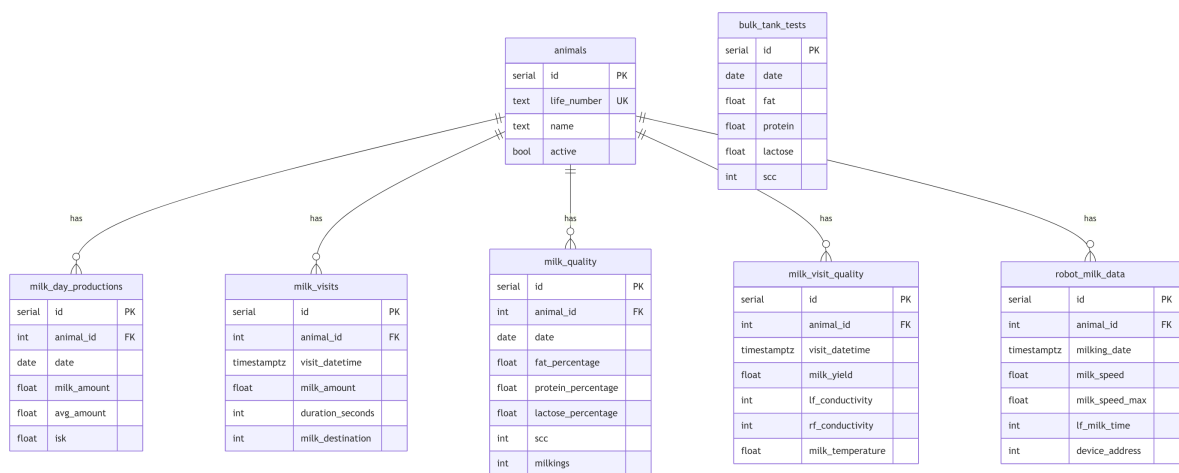


Рисунок В.3 – ER-диаграмма: доение и молоко

Кормление, здоровье и активность

На рисунке В.4 представлены таблицы кормления и мониторинга здоровья. Таблицы `feed_day_amounts`, `feed_visits`, `activities` и `ruminations` являются гипертаблицами TimescaleDB. Таблица `vet_records` содержит поля `diagnosis_code` и `confirmed`,

используемые как метки для ML-моделей.

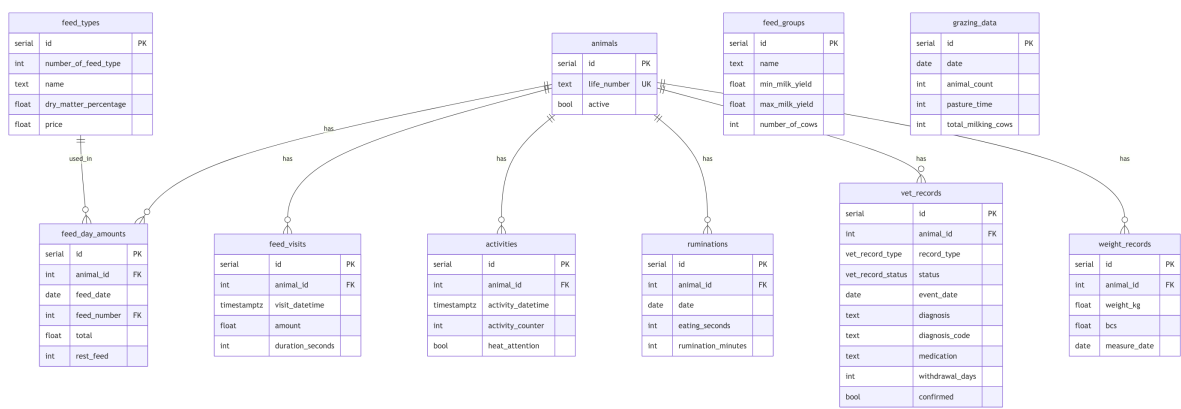


Рисунок В.4 – ER-диаграмма: кормление, здоровье и активность

Оповещения и операции

На рисунке В.5 представлены таблицы оповещений, задач, аудита, финансов и учёта запасов. Оповещения создаются автоматически движком правил `alert_engine` и доставляются через настраиваемые каналы уведомлений (браузер, Telegram).

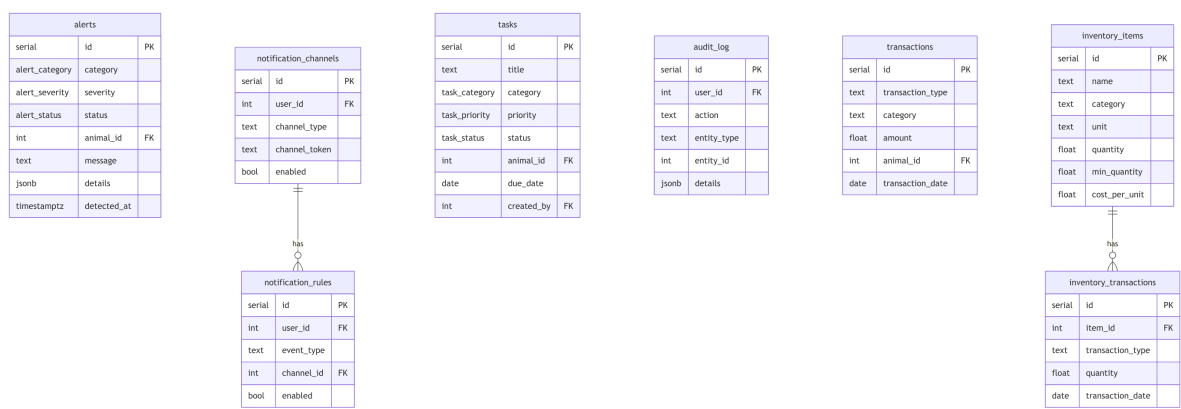


Рисунок В.5 – ER-диаграмма: оповещения и операции

Системные таблицы

На рисунке В.6 представлены системные таблицы: конфигурация интеграции с Lely с зашифрованными учётными данными, состояние синхронизации, зоны фермы и кэш погодных данных.

lely_config		
int	id	PK
bool	enabled	
text	base_url	
text	username	
text	password_encrypted	
text	farm_key_encrypted	
int	sync_interval_secs	

lely_sync_state		
serial	id	PK
text	entity_type	UK
timestampz	last_synced_at	
text	status	
bigint	records_synced	
text	error_message	

sync_log		
serial	id	PK
text	entity_type	
timestampz	last_synced_at	
text	status	
bigint	records_synced	

farm_zones		
serial	id	PK
text	name	
text	zone_type	
int	capacity	
jsonb	position_data	

weather_cache		
date	date	PK
float	temp_c	
float	humidity	
float	precipitation_mm	
float	wind_speed	
text	weather_main	
timestampz	fetches_at	

Рисунок В.6 – ER-диаграмма: системные таблицы

ПРИЛОЖЕНИЕ Г

Примеры API-запросов и ответов

В данном приложении приведены примеры основных API-запросов и ответов информационной системы. Каждый рисунок содержит запрос и соответствующий ответ сервера.

```
1 POST /api/v1/auth/login
2 Content-Type: application/json
3
4 {
5   "username": "admin",
6   "password": "securepassword"
7 }
```

```
1 200 OK
2 Set-Cookie: token=eyJhbGci...; HttpOnly; SameSite=Strict
3 Set-Cookie: refresh_token=eyJhbGci...; HttpOnly; SameSite=
   Strict
4
5 {
6   "user": {
7     "username": "admin",
8     "role": "admin",
9     "must_change_password": false
10  }
11 }
```

Рисунок Г.1 – Аутентификация: POST /api/v1/auth/login

```
1 GET /api/v1/animals?page=1&page_size=25&gender=female
2 Cookie: token=eyJhbGci...
```

```
1 200 OK
2 {
3   "items": [
4     {
5       "id": 1,
6       "life_number": "RU123456789",
7       "name": "...",
8       "gender": "female",
9       "birth_date": "2020-03-15",
10      "active": true
11    }
12  ],
13  "total": 156,
14  "page": 1,
15  "page_size": 25
16 }
```

Рисунок Г.2 – Получение списка животных: GET
/api/v1/animals


```
1 POST /api/v1/analytics/milk-trends
2 Content-Type: application/json
3
4 {
5   "animal_id": 1,
6   "days_ahead": 14
7 }
```

```
1 200 OK
2 {
3   "animal_id": 1,
4   "forecast": [
5     {
6       "date": "2026-04-21",
7       "predicted_milk_kg": 28.5,
8       "lower_bound": 25.1,
9       "upper_bound": 31.9
10    },
11    {
12      "date": "2026-04-22",
13      "predicted_milk_kg": 28.2,
14      "lower_bound": 24.8,
15      "upper_bound": 31.6
16    }
17  ],
18   "model": "holt_winters",
19   "last_actual_date": "2026-04-20"
20 }
```

Рисунок Г.3 – Прогноз надоев: POST
/api/v1/analytics/milk-trends

```
1 POST /predict/mastitis
2 Content-Type: application/json
3 X-API-Key: <api_key>
4
5 {
6   "animal_id": 1,
7   "scc": 350000,
8   "conductivity": 8.5,
9   "fpr": 1.45,
10  "milk_change_pct": -15.0,
11  "dim": 45,
12  "lactation": 2
13 }
```

```
1 200 OK
2 {
3   "animal_id": 1,
4   "risk_level": "high",
5   "probability": 0.87,
6   "contributing_factors": [
7     {"factor": "scc_change", "importance": 0.35},
8     {"factor": "conductivity", "importance": 0.25},
9     {"factor": "milk_change", "importance": 0.20}
10  ]
11 }
```

Рисунок Г.4 – Обнаружение мастита: POST /predict/mastitis

```
1 GET /api/v1/reports/export/R16/csv?date_from=2026-04-01
2 Cookie: token=eyJhbGci...
```

```
1 200 OK
2 Content-Type: text/csv
3 Content-Disposition: attachment; filename="R16_2026-04-20.csv"
4
5 date,total_milk_kg,avg_milk_kg,avg_fat_pct
6 2026-04-01,4350.0,29.0,3.85
7 2026-04-02,4280.5,28.5,3.82
```

Рисунок Г.5 – Экспорт отчёта в CSV: GET
/api/v1/reports/export